

Registro de Decisiones de Arquitectura (ADR) - TFG

Este documento centraliza las decisiones técnicas críticas tomadas durante el desarrollo del sistema, justificando su contexto, las alternativas evaluadas y las consecuencias en el diseño de software.

[ADR-01] React + Spring Boot como Arquitectura Desacoplada

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Se requiere construir una plataforma web de formación con capacidad de crecimiento funcional, garantizando que el diseño visual y la lógica de negocio no interfieran entre sí.
 - **Decisión:** Adoptar una arquitectura cliente-servidor completamente desacoplada mediante una Single Page Application (SPA) en React y una API REST empresarial en Spring Boot.
 - **Justificación para el TFG:** Permite la separación estricta de responsabilidades (Frontend/Backend). Facilita el desarrollo independiente y cumple con los estándares actuales de la industria del software.
 - **Consecuencias:** Duplicidad inicial de modelos de datos en TypeScript y Java, pero total libertad para rediseñar la interfaz sin alterar la base de datos.
-

[ADR-02] Organización del Espacio de Trabajo mediante Monorepo

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** El proyecto involucra dos tecnologías independientes (React y Spring Boot) gestionadas por un único desarrollador durante el ciclo de vida del TFG.
 - **Decisión:** Alojar ambas aplicaciones en directorios diferenciados (**frontend/** y **backend/**) dentro del mismo repositorio de Git.
 - **Justificación para el TFG:** Simplifica drásticamente la gestión del código, reduce la fricción operativa y permite realizar un único *commit* que relacione un cambio funcional simultáneo en el cliente y el servidor.
 - **Consecuencias:** El repositorio crece en tamaño, pero se centraliza toda la documentación técnica en la raíz del proyecto de cara a la evaluación del tribunal.
-

[ADR-03] Transición por Fases de Sesión HTTP a JSON Web Tokens (JWT)

- **Fecha:** Junio 2026
- **Estatus:** En Progreso (Fase 1)
- **Contexto:** El backend opera mediante autenticación tradicional basada en sesión, lo que acopla el servidor al estado del cliente y limita la escalabilidad de la API REST.
- **Decisión:** Migrar el sistema hacia un modelo de seguridad basado en tokens JWT (*stateless*), utilizando una estrategia de convivencia temporal en dos capas.
- **Justificación para el TFG:** La arquitectura *stateless* es el estándar nativo para APIs REST. La migración por fases mitiga el riesgo de regresiones y fallos en cascada, demostrando la aplicación de buenas prácticas de control de riesgos.

- **Consecuencias:** Durante la transición, el filtro de seguridad debe ser "no intrusivo" para permitir que convivan clientes con cookies y clientes con cabeceras **Bearer**, aumentando temporalmente la complejidad de **SecurityConfig.java**.
-

[ADR-04] Configuración de Propiedades JWT y Fallbacks de Entorno para Tests

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Al introducir el componente de configuración estricta **JwtProperties**, el entorno de pruebas unitarias (**BackendApplicationTests**) fallaba críticamente (**BUILD FAILURE**) debido a que la propiedad requerida **app.jwt.secret** no se encontraba definida en el contexto de carga de los tests, provocando una excepción de tipo **IllegalStateException**.
 - **Decisión:** Incorporar valores base estandarizados y mecanismos de *fallback* para desarrollo y pruebas dentro del archivo global **application.properties**, desacoplándolo temporalmente de la configuración rígida por variables de entorno del sistema.
 - **Justificación para el TFG:** Demuestra la aplicación del principio de robustez en el ciclo de vida del software. Al garantizar un valor por defecto seguro en entornos no productivos, se asegura que el ecosistema de integración y pruebas unitarias sea independiente del sistema operativo o máquina donde el tribunal o evaluador compile el proyecto.
 - **Consecuencias:** Se recupera la estabilidad del contexto de Spring Boot en un estado saludable (**BUILD SUCCESS**), permitiendo avanzar en la inyección de dependencias de seguridad de forma controlada y aislada.
-

[ADR-05] Implementación del Modelo de Autenticación Híbrido (Sesión + JWT)

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** La migración desde un sistema basado en sesión con estado (*stateful*) hacia una arquitectura basada en tokens criptográficos (*stateless*) introduce riesgos de ruptura en el frontend de React. Se requiere que el servidor admita la emisión y validación de tokens sin inhabilitar el flujo operativo preexistente.
 - **Decisión:** Desarrollar e integrar una capa híbrida que permita la coexistencia de ambos mecanismos de autenticación mediante la inyección pasiva de un filtro personalizado (**JwtAuthenticationFilter**) en la cadena de seguridad de Spring.
 - **Justificación para el TFG:** Demuestra la aplicación de metodologías de control de riesgos y desarrollo ágil incremental. Al unificar la entidad **Users** con la interfaz **UserDetails** de Spring Security, se logra que la resolución de la identidad del usuario a través del contexto (**SecurityContextHolder**) sea agnóstica al origen del acceso. Esto permite dotar al sistema de una doble vía de entrada completamente desacoplada.
 - **Consecuencias:** Se mantiene la estabilidad operativa del sistema en un estado saludable (**BUILD SUCCESS**), permitiendo que el cliente en React migre de forma progresiva a la cabecera **Authorization Bearer** sin perder la sesión clásica como mecanismo de respaldo automático (*fallback*).
-

[ADR-06] Gestión de Estado Global y Persistencia de Tokens en React

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Al adaptar la arquitectura del cliente en React para procesar la nueva respuesta híbrida del servidor, se requería una estrategia de persistencia en el navegador que permitiese almacenar tanto los datos descriptivos del perfil del usuario (para la renderización de la interfaz) como el token criptográfico JWT (para la autorización de red) sin generar acoplamiento ni vulnerabilidades de seguridad por exposición de credenciales.
 - **Decisión:** Modificar el ecosistema del contexto de autenticación (`authTypes.ts`, `AuthContext.tsx`, `AuthProvider.tsx`) y rediseñar el módulo de almacenamiento local (`authStorage.ts`) para separar físicamente la persistencia del objeto `AuthUser` respecto al string del `accessToken`.
 - **Justificación para el TFG:** Esta separación de responsabilidades en el almacenamiento local (`localStorage`) es una buena práctica de ingeniería web fundamental para la memoria del TFG. Aislar el token en su propia clave de memoria (`accessToken`) independiza la lógica de las llamadas HTTP respecto al estado visual de la interfaz. Esto facilitará que en la siguiente fase de desarrollo se pueda inyectar el token de forma automatizada en las cabeceras `Authorization Bearer` de un cliente Axios o Fetch centralizado, manteniendo intacto el ciclo de renderizado de los componentes de React.
 - **Consecuencias:** Se estabiliza el tipado estático del frontend sin advertencias del compilador de TypeScript, asegurando un punto de partida óptimo para conectar las llamadas API reales de los formularios de la SPA sin alterar la experiencia de usuario preexistente.
-

[ADR-07] Consumo del Contrato JWT y Desacoplamiento de Vistas en el Cliente

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Tras rediseñar el almacenamiento de tokens en React, se requería adaptar la interfaz gráfica de inicio de sesión (`AuthModal.tsx`) para consumir el nuevo payload del servidor sin alterar el diseño visual, las animaciones ni la experiencia de usuario de los formularios de la SPA.
 - **Decisión:** Refactorizar la función controladora del formulario de Login para realizar el tipado estático seguro (*type casting*) de la respuesta HTTP de Axios hacia la interfaz `AuthTokenResponse`, delegando el almacenamiento inmediato del token de acceso de red y los metadatos de perfil en el storage local y actualizando el estado de autenticación de la aplicación de forma transparente.
 - **Justificación para el TFG:** Demuestra la capacidad de consumir APIs bajo contratos de tipado estricto en entornos corporativos. Al desligar el procesamiento del payload JSON de la capa de presentación visual, se garantiza que los componentes gráficos permanezcan aislados frente a futuras modificaciones en la estructura de las propiedades devueltas por Spring Boot.
 - **Consecuencias:** Se logra un flujo de autenticación seguro, tipado y completamente operativo que actualiza el estado reactivo global del cliente de manera síncrona tras la verificación de las credenciales en PostgreSQL.
-

[ADR-08] Centralización de Peticiones HTTP mediante Interceptores de Red

- **Fecha:** Junio 2026

- **Estatus:** Aceptado
 - **Contexto:** Almacenado el token JWT en el cliente, se requería un mecanismo para adjuntarlo en la cabecera **Authorization: Bearer** de cada petición hacia recursos protegidos del backend, evitando que los componentes visuales asuman lógica repetitiva de red o manipulen directamente el almacenamiento físico.
 - **Decisión:** Implementar un cliente HTTP personalizado (**src/services/apiClient.ts**) basado en Axios, de uso obligatorio para las comunicaciones con la API, incorporando interceptores automáticos de solicitud (*Request Interceptors*).
 - **Justificación para el TFG:** Demuestra un profundo conocimiento del patrón de diseño estructural *Proxy* o *Middleware* aplicado a comunicaciones de red. Al delegar la inyección del token en un interceptor centralizado, se garantiza el principio de Securitización Transparente: el resto de la aplicación consume la API de forma nativa, mientras que el interceptor se encarga de auditar y adjuntar las credenciales necesarias, aislando por completo la capa visual de la capa de transporte de seguridad.
 - **Consecuencias:** Reducción drástica del acoplamiento en el frontend. Automatización completa de la inyección de seguridad en peticiones salientes y centralización de la captura de errores globales de autenticación (códigos HTTP 401).
-

[ADR-09] Mecanismo de Hidratación de Sesión mediante Validación Asíncrona del Token

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Al recargar la interfaz web de la SPA, el estado volátil de React se destruye. Se requería un mecanismo automático que determinase si el token almacenado localmente seguía siendo válido en el servidor antes de renderizar las pantallas privadas de la aplicación.
 - **Decisión:** Implementar un efecto secundario de ciclo de vida (**useEffect**) en la inicialización del **AuthProvider** para consumir el endpoint **/api/auth/me** de forma transparente, controlando el flujo mediante un estado bandera **isLoading**.
 - **Justificación para el TFG:** Demuestra el uso correcto del patrón de sincronización de estado cliente-servidor (*State Hydration*). Utilizar una variable **isLoading** evita la pérdida de sesión en recargas y unifica la verificación síncrona en el cliente con las respuestas de la API del servidor.
 - **Consecuencias:** Consolidación total de la seguridad de la SPA. Las recargas de página recuperan la identidad del usuario de forma inmediata y el estado se sincroniza de forma robusta ante tokens expirados.
-

[ADR-10] Unificación de Layouts Mediante Composición Paramétrica Reactiva al Rol

- **Fecha:** Junio 2026
- **Estatus:** Superado (Rectificado)
- **Contexto:** La arquitectura original propuesta en este registro planteaba una segregación física de archivos (**AdminLayout.tsx**, **ProfessorLayout.tsx** y **StudentLayout.tsx**) bajo la premisa de anticipar necesidades futuras de "composición especializada". Sin embargo, tras una auditoría técnica orientada al despliegue, se evidenció un escenario de sobreingeniería y deuda técnica: los tres

componentes compartían el 98% de su infraestructura lógica y directivas estéticas de Tailwind CSS, divergiendo únicamente en parámetros milimétricos de espaciado lateral (`px-4` frente a `px-6`). Mantener esta dispersión multiplicaba el riesgo de desincronización ante actualizaciones globales del diseño perimetral.

- **Decisión:** Deprecar y eliminar del sistema los tres archivos redundantes de diseño por rol. Toda la infraestructura visual del frontend se centraliza de manera absoluta en un único componente inteligente: `DashboardLayout.tsx`. Este elemento se redefine como un contenedor paramétrico que autodetecta en tiempo real el rol del usuario autenticado a través del contexto global de `useAuth`, inyectando dinámicamente las clases geométricas correspondientes.
- **Justificación para el TFG:** Esta rectificación prioriza de forma rigurosa el principio fundamental *DRY* (*Don't Repeat Yourself*) sobre asunciones de diseño tempranas y desacopladas de la realidad del código fuente. Al consolidar la interfaz perimetral en un único punto de control, se optimiza la mantenibilidad del software. Cualquier modificación estructural (como la futura adición de pies de página o menús laterales) requiere una única edición centralizada, reduciendo a cero el radio de fallo por omisión en vistas satélite y eliminando la redundancia estructural en el árbol de componentes de React.
- **Consecuencias:**
 - **Mitigación de Deuda Técnica:** Reducción drástica del volumen de archivos huérfanos de lógica propia en el directorio de layouts, simplificando la auditoría de código del frontend.
 - **Coherencia Geométrica Garantizada:** La paridad visual entre los paneles de control queda blindada de forma nativa por el compilador, aplicando los espaciados específicos reactivos únicamente bajo la condición estricta de jerarquía (como el perfil del administrador).
 - **Optimización del Enrutamiento:** Simplificación en la integración de las vistas del cliente (`StudentDashboard`, `StudentProfilePage`, etc.), las cuales migran hacia el consumo de un único envoltorio homogéneo, facilitando el mantenimiento de las importaciones y la legibilidad en los módulos de navegación.

[ADR-11] Mitigación de Desincronización Temporal mediante Margen de Tolerancia (Clock Skew) en JWT

Estado

Aceptado

Contexto

En arquitecturas distribuidas de producción, los relojes internos de los diferentes entornos (servidor API de Spring Boot, base de datos PostgreSQL y clientes web/móviles) pueden sufrir ligeras desincronizaciones micrométricas (desviaciones de tiempo o *clock drift*). Nuestra auditoría técnica detectó una propiedad inactiva (`app.jwt.clock-skew-seconds`) y una validación estricta de la fecha de expiración en `JwtService.java`. Esto generaba un riesgo crítico: la invalidación prematura e injustificada de tokens JWT legítimos

inmediatamente después de su emisión si el reloj del cliente o del servidor presentaba una variación de apenas unos segundos.

Decisión

Se decide desacoplar la validación de tiempo estricta e integrar de forma activa un margen de tolerancia (*Clock Skew*) en la lógica de expiración de tokens (`isExpired`) en `JwtService.java`.

Se ha parametrizado mediante inyección de dependencias (`JwtProperties`) un margen configurable de 60 segundos (valor recomendado por la especificación OAuth 2.0 / RFC 7519). Al evaluar la expiración, el tiempo absoluto actual del sistema se mitiga restándole este margen de tolerancia: `Instant adjustedCurrentTime = Instant.now(clock).minusSeconds(skew);`

Consecuencias

- **Positivas:** Se elimina el error silencioso de tokens rechazados por desalineación temporal en producción. Aumenta la robustez de la API sin degradar la experiencia de usuario.
- **Negativas:** La ventana real de validez del token se extiende exactamente por el número de segundos configurado en el *clock skew*, un impacto marginal que se asume en favor de la disponibilidad.

[ADR-12] Implementación de una Suite de Pruebas Automatizadas y Mecanismos de Smoke Testing para la Infraestructura de Seguridad Criptográfica

Fecha: Junio 2026

Estatus: Aceptado

Contexto: Una auditoría técnica interna identificó una brecha de cobertura crítica (coverage gap) en el subsistema de seguridad. El componente más sensible del servidor, `JwtService.java`, carecía de pruebas unitarias que validasen la integridad de los algoritmos de firma HS256 y la gestión de tiempos de expiración. Asimismo, el archivo predeterminado `BackendApplicationTests.java` se limitaba a una prueba de carga de contexto vacía y estéril que no aportaba valor asertivo al ciclo de vida del software, dejando la inyección de dependencias de Spring Security expuesta a errores no detectados antes del despliegue.

Decisión: Implementar una suite robusta de pruebas automatizadas utilizando JUnit 5 y Mockito. La estrategia se divide en dos niveles operativos localizados en la ruta estandarizada `src/test/java/`:

Aislamiento Criptográfico (`JwtServiceTest.java`): Creación de pruebas unitarias que emplean dobles de prueba (Mocks) para verificar exhaustivamente la emisión de tokens, la correcta extracción de claims (roles y usuarios) y la inmutabilidad temporal mediante la validación del Clock Skew.

Smoke Testing de Infraestructura: Refactorización de `BackendApplicationTests.java` para transformarlo en un test de humo activo. Este ahora valida de forma asertiva que los Beans críticos de seguridad (como el `AuthenticationManager` y los filtros JWT) se encuentren correctamente instanciados en el contenedor IoC de Spring.

Justificación para el TFG: Esta decisión garantiza el cumplimiento de los estándares profesionales de Aseguramiento de la Calidad (QA) y la correcta aplicación de la Pirámide de Pruebas. Al automatizar la validación de la lógica criptográfica, se demuestra al tribunal la capacidad de certificar el software bajo el estándar BUILD SUCCESS de Maven. El pipeline de integración alcanza una

tasa de éxito verificada del 100% (8 tests ejecutados, 0 fallos, 0 errores), proporcionando una métrica objetiva de fiabilidad técnica.

Consecuencias:

Eliminación de falsos positivos: Se erradica la posibilidad de que el sistema compile con una configuración de seguridad defectuosa.

Certificación estática: El subsistema de autenticación queda blindado contra regresiones durante futuras refactorizaciones.

Robustez en el pipeline: Se asegura que el ciclo de vida del desarrollo cuente con una red de seguridad técnica que valide la integridad de la plataforma de forma previa a cualquier despliegue en entornos de evaluación.

[ADR-13] Purificación Arquitectónica de la Capa de Presentación mediante Composición Pura e Inyección del Contenedor de Scroll Controlado

Estado

Aceptado.

Contexto

El componente transversal `GenericCard.tsx` presentaba un sobrediseño inicial que acoplaba rígidamente su estructura a propiedades de texto estáticas (como `title`, `tag`, `footerChildren`), obligando al sistema a realizar mapeos artificiales y limitando su polimorfismo. Además, la necesidad de incorporar un listado masivo de usuarios PostgreSQL en la consola del Administrador exigía un mecanismo de visualización que no degradase el DOM ni alterase de forma descontrolada el layout vertical de la aplicación ante el crecimiento de la base de datos.

Decisión

1. **Purificación UI:** Rediseñar por completo `GenericCard.tsx` eliminando todas las propiedades rígidas del contrato de TypeScript y reduciéndola a un contenedor minimalista gobernado única y exclusivamente por la propiedad nativa de composición pura `children`.
2. **Encapsulamiento del Scroll:** Crear la utilidad atómica `UserScrollList.tsx` dentro de `components/admin/` e inyectarla por composición dentro de la nueva tarjeta genérica.
3. **Contención Estricta de Altura:** Fijar una altura estricta inamovible de `h-[116px]` en el scroll y compensar simétricamente el Buscador en `AdminDashboard.tsx` mediante un acolchado geométrico equilibrado (`py-15`), garantizando un plano horizontal perfectamente alineado (50% - 50%).

Justificación para el TFG

Cumplimiento estricto del principio Abierto/Cerrado (OCP) de SOLID. La tarjeta actúa como un lienzo abstracto inmutable capaz de asimilar cualquier contenido futuro sin alterar su código fuente. La contención vertical garantiza la ergonomía visual y un consumo constante de memoria en el navegador sin importar el volumen de datos en PostgreSQL.

Consecuencias

- Erradicación total de abstracciones falsas y código muerto en el Frontend.
- Logro de simetría geométrica bidireccional perfecta en la interfaz del Administrador.
- Escalabilidad visual y de rendimiento garantizada frente a cientos de registros concurrentes.

[ADR-14] Soporte Multirrol Dinámico y Corrección de Ámbito en Bloques Asíncronos (AuthModal & NavBarUser)

- **Fecha:** Junio 2026
- **Estatus:** Aceptado
- **Contexto:** Se requería la inclusión de un nuevo rol de usuario (**PROFESSOR**) dentro de la plataforma escolar, el cual debía coexistir de manera jerárquica con los roles preexistentes (**ADMIN** y **STUDENT**). Al intentar introducir manualmente esta lógica intermedia en el cliente SPA, se produjeron fallos críticos de sintaxis: una pérdida de ámbito (Scope) en **AuthModal.tsx** por un cierre prematuro de llaves (**}}**) que desconectó los bloques **catch** y **finally** provocando el error **Parsing error: 'catch' or 'finally' expected**; una evaluación lineal incorrecta que ejecutaba siempre la ruta **/student** de forma secuencial; y una limitación estructural en JSX dentro de **NavBarUser.tsx**, ya que el operador ternario binario tradicional (**condicion ? A : B**) no admitía una estructura procedural **if/else** directa.
- **Decisión:** Saneamiento y unificación del flujo mediante dos estrategias en el frontend:
 1. Reestructurar las llaves jerárquicas del bloque de captura asíncrono en **AuthModal.tsx** y confinar la redirección dentro de una estructura de control de tres vías (**if, else if, else**) mutuamente excluyente y atómica.
 2. Implementar una función flecha auto-invocada (IIFE: **((() => { ... }()))**) dentro del árbol de renderizado JSX de **NavBarUser.tsx** para encapsular la lógica condicional múltiple. Se diseñó la identidad visual docente con un esquema esmeralda corporativo (**bg-emerald-50, text-emerald-800**) acoplado al icono descriptivo **BookOpen** de **lucide-react**.
- **Justificación para el TFG:** Demuestra el dominio de la especificación técnica de ECMAScript y la arquitectura de componentes reactivos en aplicaciones de gran envergadura (SPA). La resolución del bloqueo del árbol sintáctico del **try/catch/finally** evidencia buenas prácticas en el manejo del estado asíncrono y la robustez del software. Por su parte, la inyección de expresiones procedimentales mediante IIFE dota a la interfaz de una alta cohesión, paridad estética y mantenibilidad, criterios clave exigidos en la rúbrica de evaluación del tribunal.
- **Consecuencias:** Se eliminaron por completo los errores de compilación y de pérdida de alcance de las variables reactivas (**setError, setLoading**). La estructura condicional excluyente asegura que un usuario autenticado jamás ejecute código residual de otra ruta, mitigando redirecciones inválidas. Como deuda técnica menor, el uso de funciones auto-invocadas en JSX podría sobrecargar la legibilidad de la vista si el marcado por rol crece desmesuradamente en el futuro, planteando una potencial extracción a micro-componentes.

[ADR-15] Centralización del Control de Roles Administrativos y Optimización Stateless (Stateless Token Parsing)

- **Fecha:** Junio 2026
- **Estatus:** Aceptado

- **Contexto:** Al intentar implementar la funcionalidad de alteración de privilegios en el componente del frontend `AdminDashboard.tsx`, el servidor Spring Boot emitía rechazos asíncronos y revertía localmente las mutaciones a su estado por defecto (`STUDENT`). Este fallo se debía a tres factores críticos: la omisión del verbo HTTP `PATCH` en las políticas CORS globales, la falta de mapeo explícito de la subruta de gestión `/api/auth/users/**` en las reglas de Spring Security, y un cuello de botella arquitectónico en el filtro `JwtAuthenticationFilter.java` que ejecutaba consultas redundantes a PostgreSQL (`UserDetailsService.loadUserByUsername`) en cada petición entrante.
- **Decisión:** Rediseñar e integrar la infraestructura de seguridad en tres niveles:
 1. Incorporar el método `PATCH` en el Bean de configuración CORS y mapear de manera robusta la ruta administrativa en `SecurityConfig.java` utilizando `.hasAnyAuthority("ADMIN", "ROLE_ADMIN")` para absorber discrepancias de nomenclatura de roles en bases de datos relacionales.
 2. Eliminar por completo el acceso redundante a persistencia en el ciclo de filtrado, transformando `JwtAuthenticationFilter` en un componente stateless que decodifica y reconstruye las autoridades (`SimpleGrantedAuthority`) en memoria extrayéndolas criptográficamente del token JWT (Claims).
 3. Vincular el selector interactivo `<select>` del frontend con el endpoint atómico parametrizado para salvaguardar el principio de menor privilegio.
- **Justificación para el TFG:** Aporta un valor metodológico fundamental en términos de seguridad informática y optimización de recursos. Demuestra al tribunal la capacidad de erradicar duplicidades y de transicionar con éxito desde un modelo híbrido con estado (Stateful) hacia una arquitectura puramente desacoplada (Stateless / RESTful). El parseo directo de claims en memoria reduce drásticamente los accesos concurrentes a la base de datos PostgreSQL, mejorando los tiempos de respuesta y la escalabilidad de la plataforma frente a picos de tráfico simulados.
- **Consecuencias:** Se elimina la degradación de rendimiento por dobles lecturas a disco en cada petición asegurada. El Administrador puede gestionar, auditar y reasignar los privilegios del alumnado y profesorado de forma instantánea y reactiva en la interfaz de usuario, garantizando una consistencia de datos atómica y libre de bloqueos cruzados en el servidor de preflight de los navegadores.

[ADR-16] Unificación del Perfil Universal Atómico y Desacoplamiento de Flujos de Redirección mediante Cohesión de Componentes y Sincronización Síncrona del Contexto JWT

- **Fecha:** Junio 2026
- **Estatus:** Aceptado
- **Contexto:** Gestión de Perfil de Usuario (STUDENT, PROFESSOR, ADMIN) e Infraestructura de Navegación SPA.
- **Autores:** Luis (Desarrollador) / Laura / Jorge / Tutor TFG

1. Contexto y Problema

El sistema presentaba dos desafíos críticos interconectados que afectaban la experiencia de usuario y la integridad de los datos entre React (Vite, TSX) y el backend (Spring Boot 3 + PostgreSQL):

1. **Cortocircuito de Persistencia Backend (Excepción Hibernate):** El filtro de seguridad (`JwtAuthenticationFilter`) inyectaba un objeto `UserDetails` desvinculado de la sesión activa de

Hibernate (*Detached*). Al enviar un payload `PUT` genérico (`/api/v1/profile/update`) para actualizar datos del perfil, Hibernate lanzaba sistemáticamente la excepción de infraestructura `org.hibernate.AssertionFailure: null identifier (UserProfile)`. El mecanismo `@MapsId` fallaba al derivar la clave de la entidad débil `user_profiles` debido al estado transaccional inestable de la entidad fuerte `users` en transacciones de actualización concurrentes.

2. **Bloqueo y Rebote Asíncrono en el Frontend (Navegación SPA):** Al interactuar con el elemento de interfaz de usuario (`GenericButton`) que muestra el nombre del estudiante autenticado ("Luis") en el componente de cabecera, la aplicación omitía el callback de navegación y bloqueaba el acceso de forma perenne. Se detectaron tres anomalías: una discrepancia semántica en TypeScript entre manejadores `MouseEventHandler` y funciones puras `() => void` bajo la directiva `verbatimModuleSyntax`, un acoplamiento innecesario por propagación vertical de propiedades (`Props Drilling`) desde `MainNavbar.tsx` hacia `NavbarUser.tsx`, y un rebote asíncrono destructivo en el guardia de seguridad (`ProtectedRoute.tsx`), el cual interpretaba el estado transitorio inicializado en `null` de la sesión como un falso negativo, expulsando al usuario debido a la regla comodín (`path="*"`) antes de terminar la hidratación del token JWT.

2. Decisión Arquitectónica

Para garantizar un diseño de software robusto, limpio y genérico alineado con las exigencias metodológicas del TFG, se rechazó cualquier duplicidad de endpoints o consultas nativas, implementando una estrategia estructural doble:

1. **Sincronización Avanzada de Persistencia Backend:** Inyectar la instrucción `userRepository.saveAndFlush(currentUser)` en la cabecera del procesamiento del controlador. Esto obliga a Spring Data JPA a capturar la entidad desacoplada, integrarla en la sesión activa (`EntityManager.merge()`) y vaciar los cambios sincronamente. Se invierte el orden físico, ejecutando primero `profileRepository.save(profile)` y luego `userRepository.save(currentUser)` para estabilizar el mecanismo `@MapsId`.
2. **Simetría y Composición por Roles en el Frontend:** Crear páginas contenedoras por rol (`StudentProfilePage`, `ProfessorProfilePage`, `AdminProfilePage`) compartiendo por composición el mismo formulario dinámico en `ProfileSettings.tsx`. El Navbar redirige correctamente mediante un mapeo de rutas según el rol.
3. **Autonomía Contextual y Sincronización JWT:** Dotar a `NavbarUser.tsx` de plena autonomía contextual consumiendo directamente el hook global de autenticación (`useAuth`). Se implementa la función flecha `() => handleProfileRedirect()` para aislar conflictos de firmas de tipos, y se sustituye el enrutamiento virtual por una navegación imperativa física con `window.location.href`. Esto fuerza un ciclo de hidratación limpio desde la raíz que obliga al guardián a respetar el estado preventivo (`isLoading`) mientras se recupera el token del almacenamiento local.

3. Justificación para el TFG

Aporta un valor metodológico fundamental en el área de ingeniería de interfaces y patrones de arquitectura de software para Single Page Applications (SPA). Demuestra al tribunal el dominio práctico sobre el principio de **Cohesión y Autonomía de Componentes**, erradicando el acoplamiento rígido de herencia de callbacks.

En el backend, expone la capacidad de gestionar con éxito transacciones atómicas seguras y el ciclo de vida de entidades JPA acopladas (1:1) mediante claves compartidas en PostgreSQL.

4. Consecuencias

- **Positivas:** Se elimina la transferencia de propiedades muertas y las advertencias del linter (`eslint`). El flujo de acceso se homologa con el mecanismo nativo del módulo de inicio de sesión (`AuthModal.tsx`). El endpoint genérico da soporte transparente e idéntico a los tres roles (Luis, Laura y Jorge). Se habilitó con éxito la subida de avatares mediante `Multipart/Form-Data` con identificadores UUID únicos. Además, se resolvió la deuda técnica del scroll lateral en la consola de administración (`UserScrollList.tsx`) indexando los badges en un diccionario estático y aplicando una clave compuesta triple (`key={item.userId || item.username || ...}`) que fulmina el warning de llaves duplicadas o indefinidas en React.
 - **Negativas / Deuda Técnica:** Ninguna detectada. El circuito de administración, enrutamiento y perfil universal queda cerrado y validado al 100%.
-

[ADR-17] Arquitectura del Módulo de Gestión de Cursos y Modelado de Relaciones Unidireccionales para el Perfil Profesor

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Circuito del Profesor (Laura) - Creación y gestión de contenidos académicas.
 - **Autores:** Luis (Desarrollador) / Tutor TFG
-

1. Contexto y Problema

Con el perfil unificado funcionando, el sistema requería implementar el circuito del profesor (Laura) para permitir la creación, edición y lectura de cursos en PostgreSQL. El desafío técnico radicaba en modelar las relaciones en Spring Boot 3 de forma **unidireccional** (para evitar referencias circulares e infinitas al serializar con Jackson a JSON) y asegurar que solo los usuarios con el rol **PROFESSOR** tengan privilegios de mutación en estos endpoints.

2. Decisión Arquitectónica

Se implementaron las siguientes directrices de diseño:

1. **Control de Acceso Basado en Roles (RBAC):** Asegurar los endpoints del controlador mediante anotaciones `@PreAuthorize("hasRole('PROFESSOR')")` a nivel de Spring Security.
 2. **Modelado Unidireccional Limpio:** La entidad `Curso` contiene la referencia al profesor (entidad `User`), pero la entidad `User` no conoce los cursos que imparte, evitando sobrecargar la memoria de la sesión de Hibernate.
 3. **Paginación e Interfaz en React:** Consumo del listado mediante Axios mapeando los estados en tarjetas escaneables dentro de `ProfessorDashboard.tsx`.
-

3. Consecuencias

- **Positivas:** Seguridad estricta en la capa de servicios (Laura puede gestionar sus cursos, pero Luis el alumno solo tiene acceso de lectura). Respuestas JSON ligeras y limpias sin necesidad de usar `@JsonManagedReference` o `@JsonIgnore`.
- **Negativas / Deuda Técnica:** Al usar relaciones unidireccionales, para obtener los cursos de un profesor específico se requiere una query personalizada en el repositorio (`findByProfessorId`), lo cual queda documentado y optimizado.

[ADR-18] Diseño de Persistencia Normalizada Multivalor y Simetría Relacional para Intereses del Alumnado

- **Fecha:** Junio 2026
- **Estatus:** Aceptado

Contexto

Al planificar la infraestructura del futuro módulo de recomendación inteligente en el frontend (`StudentDashboard.tsx`), se identificó la necesidad imperativa de capturar de forma genérica y multidimensional los intereses del estudiante en HTML/React en cinco ejes críticos: categorías temáticas, nivel, duración de cursos, idioma y subtítulos. El almacenamiento tradicional en columnas de texto plano (strings concatenados por comas) provocaría una degradación severa del rendimiento, obligando al servidor a ejecutar costosas operaciones de búsqueda de patrones con comodines (`LIKE`), anulando la utilidad de los índices y violando la Primera Forma Normal (1FN) de las bases de datos relacionales. Adicionalmente, se detectó que al abrir el componente modal multiscroll (`InterestsModal.tsx`), las preferencias no persistían marcadas en la interfaz de usuario debido a que el motor de persistencia sufría una desconexión por carga perezosa (*lazy loading*) en las colecciones secundarias y a que existía una colisión de rutas en el enrutamiento HTTP que impedía la correcta lectura del estado actual.

Decisión

Implementar un circuito de persistencia normalizado, desacoplado y simétrico estructurado en cuatro niveles técnicos:

1. **Arquitectura de Datos:** Incorporar la entidad `Interest.java` aplicando el patrón `@MapsId` en una relación `@OneToOne` con la tabla de usuarios para compartir de forma nativa la misma clave primaria (`user_id`), creando un esquema donde el ID de intereses hereda exactamente el ID del usuario en PostgreSQL y garantizando el borrado en cascada automático.
2. **Persistencia de Colecciones e Hidratación Explícita [ADR-31]:** Implementar la anotación `@ElementCollection` generando de forma transparente cinco tablas satélite indexadas de forma vertical (`interest_categories`, `interest_course_types`, `interest_durations`, `interest_languages`, y `interest_subtitle_languages`). En el método transaccional de lectura `getUserInterests` de `UserService.java`, se descarta el uso de inicializadores externos pesados y se introduce una **Lógica de Hidratación Forzada Optimizada mediante la invocación explícita del tamaño de colección (`.size()`)**. Esto destruye el proxy perezoso de Hibernate y obliga al ORM a volcar los datos reales en memoria mientras la transacción `@Transactional(readOnly = true)` se encuentra abierta, asegurando que las listas no viajen vacías al ser serializadas a JSON.

3. **Simetría en Enrutamiento API:** Reestructurar la precedencia jerárquica en `UserController.java` posicionando el endpoint estático `GET /api/auth/my-interests` de forma prioritaria antes del patrón dinámico variable `GET /{username}`. Esto anula la ambigüedad en el enrutamiento web de Spring MVC y subsana el error sintáctico de petición incorrecta (Error 400 Bad Request).
4. **Ciclo Sincronizado en Frontend:** Configurar en el modal de React un hook de efecto (`useEffect`) acoplado al estado de visibilidad (`isOpen`), invocando de manera asíncrona al cliente centralizado (`apiClient`). Los datos recuperados hidratan los estados locales (`useState`), activando de forma automática las clases CSS condicionales y los iconos de verificación (✓) de las tarjetas seleccionadas.

Justificación para el TFG

Aporta un valor metodológico fundamental en términos de diseño avanzado de bases de datos relacionales, control de enrutamiento web y optimización algorítmica de ORM. Demuestra al tribunal el cumplimiento riguroso de la teoría de normalización (1FN) al fragmentar colecciones dinámicas en casilleros elementales independientes. El orden de declaración de los métodos del controlador evidencia un dominio avanzado en el ciclo de vida de peticiones en Spring Framework. Asimismo, el control programático de la inicialización de proxies mediante llamadas `.size()` defiende la robustez y elegancia del software frente a fallos de inicialización indeterministas, eliminando acoplamientos rígidos con clases nativas del proveedor de persistencia y optimizando el ciclo de vida de la sesión transaccional. Esta simetría exacta entre el perfil de intereses del alumno y el catálogo de cursos anula la necesidad de capas de conversión intermedias en Spring Boot, lo que permitirá al futuro motor de recomendación ejecutar consultas de cruce ultra veloces mediante operaciones de conjunto indexadas (`JOIN` y cláusulas `IN`), maximizando la escalabilidad del sistema.

Consecuencias

Se elimina por completo el almacenamiento caótico de texto plano y el coste computacional de procesar expresiones regulares en disco. El estudiante puede interactuar de forma reactiva con el modal multiscroll de React para actualizar sus criterios preferentes en el mismo milisegundo. Los cambios se persisten de forma transaccional y consistente en PostgreSQL mediante un mecanismo destructivo-limpio (operaciones secuenciales de `DELETE` e `INSERT` gestionadas de forma nativa por el ORM), proporcionando una base de datos perfectamente estructurada, limpia, libre de duplicados y lista para alimentar el motor de recomendaciones con un estado HTTP 200 OK estable.

[ADR-19] Optimización Computacional del Buscador Predictivo mediante Indexación Invertida GIN, Formateo Nativo de Parámetros y Mitigación de Carga en Interfaz

- **Fecha:** Junio 2026
- **Estatus:** Aceptado

Contexto

El buscador global de cursos integrado en el panel del estudiante (`StudentDashboard.tsx`) presentaba deficiencias críticas de rendimiento y usabilidad que comprometían la escalabilidad del sistema. La consulta JPQL original implementaba una búsqueda aproximada multicampo utilizando comodines en ambos extremos del patrón (`LIKE LOWER(CONCAT('%', :keyword, '%'))`). En entornos relacionales como PostgreSQL, la presencia de un comodín inicial anula la utilidad de los índices de árbol tradicionales (B-Tree), forzando al

motor a ejecutar un escaneo secuencial completo (*Full Table Scan*) de coste computacional $O(N)$ en cada pulsación de tecla.

Adicionalmente, se detectó una anomalía de *binding* en la caché de planes de Hibernate al concatenar dinámicamente los porcentajes (%) dentro de múltiples cláusulas **OR** en JPQL. Este fenómeno cruzaba los marcadores de posición en consultas concurrentes, provocando que el motor de la base de datos interpretara la consulta de forma laxa (equivalente a **LIKE '%%'**) y devolviera registros aleatorios que no coincidían con el criterio de búsqueda. Finalmente, la ausencia de límites en la consulta volcaba todo el catálogo en la memoria RAM del servidor de Spring Boot, penalizando críticamente el hilo de renderizado del navegador al intentar dibujar cientos de componentes **GenericCard** simultáneamente.

Decisión

Implementar una reestructuración arquitectónica integral en tres capas para optimizar el flujo de datos y la eficiencia de cómputo:

1. **Capa de Persistencia (PostgreSQL):** Activar la extensión nativa **pg_trgm** y estructurar tres índices invertidos generalizados (**GIN**) basados en operaciones de trigramas (**gin_trgm_ops**) sobre las columnas críticas de búsqueda (**title**, **category**, **skills**) de la tabla **courses**. Esto fragmenta las cadenas de texto en bloques de tres caracteres, permitiendo búsquedas de coincidencia parcial indexadas de coste $O(\log N)$ u $O(1)$.
2. **Capa de Negocio y Repositorio (Spring Boot):** Modificar la firma del método en **CoursesRepository.java** eliminando las funciones de concatenación internas de JPQL. El formateo de los patrones de coincidencia aproximada (**%keyword%**) y de coincidencia inicial (**keyword%**) se traslada de forma nativa a la memoria de Java en **UserService.java**. La consulta se restringe a un lote simétrico estricto de **12 resultados** mediante **PageRequest.of(0, 12)**. Asimismo, se incorpora una cláusula algorítmica **ORDER BY CASE** en JPQL que prioriza con valor **1** los títulos que comienzan exactamente con el término buscado, con valor **2** los que lo contienen, y con valor **3** las coincidencias exclusivas por categoría o habilidades, seguidos de un ordenamiento alfabético secundario.
3. **Capa de Presentación (React & Tailwind):** Consolidar un control de tasa de peticiones (*rate limiting*) en el cliente mediante un temporizador *debounce* de 400ms acoplado al hook de efecto (**useEffect**) para evitar la saturación de peticiones HTTP en vuelo. A nivel de maquetación, se recupera la cuadrícula de tres columnas (**lg:grid-cols-3**) y se acota el contenedor exterior mediante un límite dimensional estricto de altura fija (**max-h-[290px] overflow-y-auto**), provocando de forma controlada el fenómeno de diseño **Cut-off effect** (efecto de recorte) para incentivar el scroll natural.

Justificación para el TFG

Aporta un alto valor metodológico en ingeniería de rendimiento y optimización de sistemas distribuidos. Ante el tribunal, justifica la capacidad de diagnosticar fallos indeterministas de binding en el ORM y resolverlos mediante el preformateo de cadenas en la capa de servicios de Java, garantizando que PostgreSQL reciba parámetros limpios y estrictos. El uso de la cláusula condicional **CASE** directamente en el lenguaje de consultas demuestra madurez en la delegación de lógica pesada al motor de datos en lugar de saturar la capa de aplicación en Java con bucles de ordenación tardíos.

Por último, la sincronización matemática entre el tamaño de página del backend (12 elementos) y la distribución del frontend (múltiplo exacto de la cuadrícula de 3 columnas) evidencia un diseño de software

armonizado y limpio, elevando la calidad del código a estándares de producción industrial y asegurando una respuesta de la interfaz en el orden de los milisegundos.

Consecuencias

Se erradica por completo el escaneo secuencial en disco, los falsos positivos de búsqueda por cruce de marcadores y el riesgo de desborde de buffer en el backend. Las consultas predictivas se ejecutan de forma instantánea sobre PostgreSQL incluso bajo volúmenes masivos de datos. El usuario experimenta una navegación fluida donde los criterios de ordenación semántica garantizan que los cursos más idóneos aparezcan siempre en la primera línea de visión. El componente visual queda perfectamente integrado en el espacio del dashboard, manteniendo una estética limpia, simétrica y una experiencia de usuario fluida e intuitiva libre de bloqueos en el navegador.

Enmienda A: Restricción del Alcance de Búsqueda Predictiva y Depuración de la Estructura Relacional

Fecha: Junio 2026

Estatus: Aceptado

Contexto de la Enmienda

Durante las pruebas de carga y estrés del buscador predictivo, se determinó que la inclusión de la columna `skills` (habilidades) dentro del filtro multicampo (`WHERE`) de la consulta JPQL introducía penalizaciones críticas en el tiempo de respuesta del motor de la base de datos PostgreSQL.

Técnicamente, el campo `skills` almacena un listado de tecnologías indexadas en forma de cadena plana separada por comas (ej. "Java, Spring Boot, REST"), lo que viola la **Primera Forma Normal (1FN)** del modelo relacional en este caso de uso específico. Al ejecutar operaciones de coincidencia parcial (`LIKE %keyword%`) sobre cadenas desnormalizadas y extensas, la eficiencia de la indexación por trigramas disminuye drásticamente, forzando un consumo innecesario de CPU y memoria de intercambio en disco. Adicionalmente, desde la perspectiva de la Experiencia de Usuario (UX), los retornos basados en coincidencias opacas ocultas en los metadatos de las tarjetas (no legibles a simple vista en el título o categoría) generaban confusión e indeterminismo visual en la interfaz del estudiante.

Decisión Derivada

- Refactorización del Backend (`CoursesRepository.java`):** Eliminar de forma estricta la cláusula `OR LOWER(c.skills) LIKE LOWER(:formattedKeyword)` tanto del filtro de selección como de la estructura de ponderación algorítmica `ORDER BY CASE`. La búsqueda predictiva instantánea queda restringida exclusivamente a los campos de alta densidad semántica: `title` y `category`.
- Optimización del Almacenamiento (PostgreSQL):** Ejecutar una purga del índice invertido trunco mediante el comando `DROP INDEX IF EXISTS idx_courses_skills_trgm;`, liberando espacio físico en disco y mitigando la sobrecarga computacional de reindexación en las operaciones de inserción (`INSERT`) y actualización (`UPDATE`).
- Sincronización de Interfaz (`StudentDashboard.tsx`):** Modificar los descriptores semánticos de la barra de búsqueda para transparentar el alcance real de la consulta al usuario, limitando el texto a *"por título o categoría temática"*.

4. **Aislamiento de Responsabilidad:** Reservar de forma exclusiva el atributo `skills` para el motor del **Algoritmo de Recomendación Inteligente (ADR-18)**, donde su procesamiento se ejecutará de manera asíncrona mediante un cruce de matrices frente a la tabla de intereses del estudiante, evitando penalizar el hilo de la consulta síncrona en tiempo real del catálogo.

Consecuencias

Se logra una reducción drástica en la latencia de la consulta predictiva, garantizando tiempos de respuesta estables en el orden de los milisegundos (<50ms). El repositorio y la base de datos quedan limpios de estructuras redundantes, asegurando una separación de responsabilidades (*Separation of Concerns*) perfecta entre el módulo de búsqueda global y el módulo de recomendaciones asíncronas de cara a la defensa del proyecto.

ADR-20: Refactorización del Panel del Estudiante mediante Controladores Distribuidos

Estado

Aceptado

Contexto

El componente `StudentDashboard.tsx` consolidaba en un único archivo de 428 líneas de código la lógica de consulta con debounce del buscador predictivo, la persistencia transaccional de matrículas con su respectivo plan de contingencia ante fallos de serialización de Jackson (Error 400), y el renderizado estático del panel de recomendaciones.

La previsión de crecimiento de la plataforma exige incorporar a corto plazo nuevos módulos críticos en la misma pantalla (módulo de exámenes en línea, descargas de recursos documentales, sistema de alertas de finalización y evaluación docente). Mantener el diseño monolítico original provocaría:

1. Violación del Principio de Responsabilidad Única (SRP).
2. Prop Drilling masivo al integrar nuevas interacciones.
3. Degradación del rendimiento de la interfaz debido a re-renderizados globales ante pulsaciones de teclas (*keystrokes*) en el buscador predictivo.

Decisión

Se descarta la arquitectura monolítica de UI y se adopta de forma estricta un patrón de **Controladores Distribuidos (Custom Hooks + Estado Aislado)** para segmentar el panel del estudiante.

Esta decisión se ejecuta bajo las siguientes directrices técnicas:

1. Extraer la lógica algorítmica y las mutaciones de estado HTTP a Custom Hooks independientes en archivos TypeScript plano (`.ts`).
2. Confinar el renderizado visual y los estilos de Tailwind CSS en componentes React atómicos (`.tsx`) dentro de una subcarpeta local de ámbito cerrado (`/components`).

3. Eliminar por completo el uso de estilos en línea (`style={{}}`) e inline-comments de desactivación de ESLint, sustituyéndolos por interpolación de clases de Tailwind canónicas para mantener la conformidad del linter.
4. Mantener el componente raíz `StudentDashboard.tsx` exclusivamente como un orquestador estructural de Layout y propagador de eventos mediante callbacks reactivos.

Consecuencias

Positivas

- **Alta Cohesión y Bajo Acoplamiento:** Cada dominio de negocio (Catálogo, Matrículas, Recomendaciones) se mantiene aislado e independiente.
- **Escalabilidad Lineal:** Los futuros desarrollos (exámenes, descargas, etc.) se integrarán mediante componentes paralelos autocontenidos sin riesgo de regresión sobre el código consolidado.
- **Rendimiento Optimizado:** Las actualizaciones de estado del buscador predictivo quedan confinadas en el subárbol de `CourseCatalog.tsx`, evitando ciclos de cómputo y renderizado en el panel de asignaturas o en la barra lateral.
- **Encapsulamiento del Error 400:** El plan de hidratación resiliente local frente al bug de Jackson queda estrictamente encapsulado en el ecosistema del hook de asignaturas en curso.

Negativas

- **Incremento en el Volumen de Archivos:** El panel del estudiante pasa de constar de un único archivo a estructurarse en un conjunto distribuido de 5 archivos especializados.

[ADR-21] Contrato de Expiración del Token con Tipado Estricto (TTL Centralizado)

- **Fecha:** Junio 2026
- **Estatus:** Aceptado
- **Contexto:** Existía una inconsistencia crítica en la gestión del tiempo de vida de la sesión (*Time-To-Live* o TTL) entre ambas capas de la aplicación. El backend calculaba la expiración de forma correcta, pero el frontend aplicaba parches de maquetación de tipos mediante un casting forzado a `unknown` para acceder a una propiedad inexistente denominada `expiresInSeconds`. Al fallar esta lectura por discrepancia de contratos, el sistema activaba de forma silenciosa un temporizador de emergencia fijo de 15 minutos, lo que comprometía la integridad del sistema de autenticación y provocaba desconexiones prematuras o estados inconsistentes en la interfaz de usuario.
- **Decisión:** Estandarizar de forma estricta el contrato de transferencia de datos eliminando los artificios de tipado condicional en el archivo `AuthProvider.tsx`. El campo `expiresIn` se mapea ahora de manera homogénea entre el registro de Java del servidor y su correspondiente interfaz en TypeScript como un tipo numérico estricto. El cliente calcula el instante de expiración consumiendo directamente este valor sin transformaciones ambiguas ni escapes en el compilador.
- **Justificación para el TFG:** Sigue de manera rigurosa los principios de diseño guiado por contratos en arquitecturas desacopladas cliente/servidor. Al forzar una paridad absoluta entre los tipos del backend y el frontend, se elimina la deuda técnica y se garantiza que el flujo de control matemático del TTL sea exacto. Esto evita que el compilador ignore discrepancias en las estructuras de datos compartidas y eleva la robustez perimetral del sistema de sesiones.

- **Consecuencias:**
 - **Consistencia Operacional Total:** Eliminación definitiva de los errores de desincronización en la sesión y pantallas congeladas, asegurando que el ciclo de vida del usuario en el navegador coincida exactamente con la validez del token en el servidor.
 - **Flujo de Persistencia Seguro:** Garantía de un tipado seguro en todo el flujo de autenticación, blindando la integridad de los datos desde la respuesta HTTP original hasta su almacenamiento en las capas de persistencia local (*localStorage*).
-

[ADR-22] Externalización y Seguridad de Políticas CORS mediante Variables de Entorno

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** La configuración de la seguridad perimetral en la clase `SecurityConfig.java` del backend restringía las solicitudes entrantes permitiendo únicamente el origen estático `http://localhost:5173`. Esta codificación rígida (*hardcodeada*) en el código fuente constituía un acoplamiento crítico con el entorno de desarrollo local, lo que impedía por completo el despliegue del sistema en servidores productivos reales bajo políticas de navegador (*Cross-Origin Resource Sharing*) a menos que se realizaran modificaciones manuales propensas a errores antes de compilar.
 - **Decisión:** Migrar la directiva de orígenes permitidos de CORS hacia un modelo de inyección de dependencias dinámica en Spring Boot. Se implementa el uso de la anotación `@Value` ligada a una variable de entorno del sistema, configurando un mecanismo de respaldo (*fallback*) adaptativo que emplea la ruta local por defecto si la variable externa se encuentra ausente en el sistema operativo.
 - **Justificación para el TFG:** Cumple de manera rigurosa con los principios de portabilidad y desacoplamiento de la metodología de las *Twelve-Factor Apps* para sistemas nativos de la nube. Al externalizar las directrices de red del código compilado, el archivo de seguridad de Spring Security queda cerrado a modificaciones físicas de infraestructura, permitiendo canalizar despliegues automáticos y parametrizables en entornos de desarrollo, pruebas o producción sin alterar un solo byte del artefacto compilado.
 - **Consecuencias:**
 - **Portabilidad y Automatización:** Mejora inmediata en la portabilidad del sistema, permitiendo su despliegue inmediato en cualquier proveedor PaaS o la nube (como Render, AWS o Heroku) configurando la dirección web del frontend de forma puramente operativa.
 - **Seguridad y Limpieza del IDE:** Cumplimiento de las mejores prácticas de seguridad operativa, evitando la exposición de configuraciones locales y silenciando las advertencias o avisos amarillos del editor de código en el archivo `application.properties` al delegar el flujo de datos sobre variables del entorno.
-

[ADR-23] Validación Perimetral de Archivos y Mitigación de Ataques RCE

- **Fecha:** Junio 2026
- **Estatus:** Aceptado
- **Contexto:** El servicio de almacenamiento en el servidor permitía la subida y persistencia de recursos en disco limitando únicamente el tamaño máximo de los ficheros a 5MB a través de la directiva del servlet. No obstante, la ausencia total de validaciones sobre el tipo de contenido representaba una vulnerabilidad crítica de seguridad. Un atacante autenticado o un usuario malintencionado podría

evadir el propósito de la funcionalidad cargando scripts ejecutables (como shells o scripts `.php`), obteniendo la capacidad de comprometer el servidor mediante la ejecución remota de código (*Remote Code Execution* o RCE).

- **Decisión:** Implementar un mecanismo imperativo de validación perimetral dual dentro de la clase `FileStorageService.java`. Antes de escribir cualquier flujo de datos en el disco, el sistema extrae de forma segura la extensión real del nombre del fichero y valida la cabecera `Content-Type` (*MIME Type*). Esta restricción se aplica mediante una arquitectura segregada por el contexto de la carpeta de destino: la subcarpeta `avatars` acepta estrictamente formatos gráficos autorizados (`.jpg`, `.jpeg`, `.png`, `.webp`), mientras que la subcarpeta `documents` se restringe de forma exclusiva a extensiones `.pdf`.
- **Justificación para el TFG:** Sigue las directrices de la guía OWASP y el principio de defensa en profundidad en sistemas web. Validar de forma combinada la extensión y el tipo MIME mitiga los ataques de suplantación de identidad de archivos (*MIME-sniffing*). Cualquier intento de alteración o desajuste con las listas blancas de formatos permitidos detiene la transacción de inmediato lanzando una excepción controlada, impidiendo que material potencialmente destructivo alcance el sistema de archivos local.
- **Consecuencias:**
 - **Inmunización frente a RCE:** Blindaje absoluto del servidor ante la persistencia de archivos maliciosos, neutralizando vectores de ataque orientados al secuestro de recursos o ejecución de comandos en el host.
 - **Centralización de Reglas de Negocio:** Los controladores de la aplicación (como `ProfileController.java`) delegan la responsabilidad del control de formatos en el servicio de almacenamiento, garantizando que cualquier nuevo módulo de subida en el futuro herede nativamente estas directivas de seguridad.

[ADR-24] Gestión de Sesión Robusta mediante la Visibility API del Navegador

- **Fecha:** Junio 2026
- **Estatus:** Aceptado
- **Contexto:** La desconexión proactiva por tiempo en el frontend delegaba el control de la expiración de la sesión en un temporizador lineal `setTimeout` en el cliente. Sin embargo, los navegadores web modernos (como Chrome, Edge o Safari) aplican políticas agresivas de ahorro de energía que suspenden o ralentizan los hilos de temporizadores en pestañas inactivas o en segundo plano. Esto provocaba que el reloj de la aplicación se detuviera, generando "sesiones fantasma" donde el usuario regresaba horas después viendo una interfaz falsamente autenticada que solo fallaba al intentar interactuar y colisionar contra el backend.
- **Decisión:** Migrar la lógica de desconexión proactiva en el archivo `AuthProvider.tsx` para utilizar un enfoque basado en eventos de ciclo de vida del navegador a través de la *Visibility API*. Se implementa la escucha del evento nativo `visibilitychange` combinado con una rutina ligera de muestreo cíclico de alta frecuencia (`setInterval` cada 5 segundos). En el momento exacto en que el usuario reactiva o enfoca la pestaña, el sistema ejecuta una auditoría de tiempo inmediata recalculando la validez del token frente al tiempo real del sistema operativo (`Date.now() >= expiresAt`).
- **Justificación para el TFG:** Demuestra una comprensión avanzada del entorno de ejecución asíncrono en los motores de renderizado de JavaScript y las limitaciones físicas del hardware actual. En lugar de confiar ciegamente en temporizadores lineales vulnerables a la congelación del navegador, el frontend

se vuelve autoconsciente de su estado de visualización, blindando el perímetro de autenticación en el cliente y garantizando la sincronización con el tiempo absoluto de expiración definido por el servidor.

- **Consecuencias:**
 - **Eliminación de Sesiones Fantasma:** Cierre de sesión preciso e inmediato en el milisegundo en que el usuario regresa a la aplicación, impidiendo la exposición visual de datos sensibles en interfaces obsoletas tras periodos prolongados de inactividad.
 - **Consistencia de Estado:** Desaparición de estados de autenticación inconsistentes en la interfaz de usuario, mejorando la experiencia del estudiante al sincronizar de forma determinista el estado de la SPA con el ciclo de vida real del token JWT.
-

[ADR-25] Normalización Semántica de Errores en la API REST

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Múltiples operaciones críticas en la capa de servicios del backend (como en la clase `UserService.java`) recurrían al lanzamiento directo de la excepción genérica `RuntimeException` al no localizar registros en la base de datos o ante fallos de validación. Aunque el servidor dispone de un componente interceptor centralizado (`GlobalExceptionHandler.java`), el uso de excepciones sin tipar impedía que el manejador pudiera discernir el origen del fallo, forzando a que el sistema procesara de forma homogénea cualquier anomalía bajo el mismo código genérico HTTP 500 (*Internal Server Error*), lo que deterioraba gravemente el rigor semántico y la claridad de la API REST.
 - **Decisión:** Desterrar por completo el uso de excepciones genéricas e incorporar una arquitectura de excepciones semánticas personalizadas de grano fino. Se procede a la adopción y sobrecarga de clases específicas como `ResourceNotFoundException` y `UserAlreadyExistsException`. Estas excepciones se vinculan directamente a códigos de estado HTTP específicos (`404 Not Found` y `409 Conflict`, respectivamente) utilizando las directivas de Spring Boot y el mapeo de respuestas estructuradas.
 - **Justificación para el TFG:** Sigue de manera rigurosa las especificaciones del protocolo HTTP y las mejores prácticas en el diseño de arquitecturas RESTful profesionales. Al dotar a la capa de servicios de la capacidad de comunicar anomalías de datos con nombres propios, el interceptor global puede traducir el fallo en un objeto JSON homogéneo y con el código de estado correspondiente. Esto previene fugas de información del sistema en las trazas de error y proporciona un contrato predecible para el consumo del cliente.
 - **Consecuencias:**
 - **API REST Semántica y Estándar:** Las peticiones por recursos inexistentes o conflictos de datos devuelven códigos HTTP semánticos (404 y 409) limpios en lugar de alarmantes y opacos errores internos 500, profesionalizando la interfaz del servidor.
 - **Facilidad de Depuración e Integración:** Proporciona al frontend mensajes de error claros, estructurados y consistentes, agilizando el diagnóstico en fases de pruebas y mejorando drásticamente la comunicación entre las capas del sistema mediante el uso correcto de los estándares de la industria.
-

[ADR-26] Estrategia de Aseguramiento de Calidad Híbrida y Cobertura de Regresiones en Backend y Frontend

- **Fecha:** Junio 2026
- **Estatus:** Aceptado

- **Contexto:** Tras las profundas refactorizaciones perimetrales y de seguridad realizadas en el sistema (unificación de layouts, inyección dinámica de CORS, validación dual de archivos y control de sesiones mediante la Visibility API), la suite de pruebas automatizadas se encontraba desactualizada y desalineada con la arquitectura de la plataforma. El sistema adolecía de puntos ciegos (*blind spots*) críticos de extremo a extremo: el backend no auditaba el rechazo de scripts maliciosos de tipo RCE en la capa de almacenamiento ni validaba las políticas de red frente a orígenes no autorizados en los filtros de seguridad, mientras que el frontend no comprobaba la reactividad del cliente ante la congelación de hilos de tiempo del navegador.
- **Decisión:** Diseñar e implementar una suite de pruebas automatizadas integral utilizando una estrategia híbrida y complementaria para ambas capas del software.
 1. Para el **Backend (Spring Boot)**, se implementan pruebas unitarias y de integración mediante JUnit 5 y Mockito; además, se adopta la simulación de servlets mediante entornos de contexto web inyectados dinámicamente (**MockMvcBuilders** acoplado al filtro de seguridad) para blindar perimetralmente la persistencia, mitigar fallos de infraestructura y validar que la inyección de propiedades externas para CORS actúa correctamente interceptando peticiones maliciosas (OPTIONS / Preflight).
 2. Para el **Frontend (React)**, se estructura una arquitectura de pruebas de comportamiento aisladas mediante Vitest y React Testing Library, recurriendo al mockeo semántico de componentes e hilos de tiempo falsos (**vi.useFakeTimers**) para evaluar las directivas de layouts paramétricos y los eventos de ciclo de vida del navegador.
- **Justificación para el TFG:** Aporta el máximo nivel de rigor metodológico y madurez de ingeniería de software exigido en un TFG. En lugar de limitarse a pruebas aisladas y superficiales, la suite valida la integración real entre las capas de negocio, seguridad, red y persistencia de datos. El uso de maquetación de contexto web simulado en Spring Boot y mocks controlados en React aísla las responsabilidades de los componentes, garantizando que los tests actúen como un escudo perimetral inmune a futuras alteraciones en la base de datos o modificaciones estéticas en la interfaz de usuario.
- **Consecuencias:**
 - **Inmunización Dual ante Regresiones:** El sistema queda blindado contra fallos colaterales en ambas capas, asegurando con un 100% de certeza técnica verificada por consola (**BUILD SUCCESS** en Spring Boot y **PASS** en Vitest) que el servidor rechaza scripts maliciosos, los filtros de red bloquean orígenes no autorizados y el cliente destruye proactivamente las sesiones fantasma en milisegundos.
 - **Documentación Ejecutable Homogénea:** La suite de pruebas actúa como documentación viva y simétrica del comportamiento esperado de la plataforma de desarrollo, facilitando el diagnóstico en fases de mantenimiento y garantizando la estabilidad operativa global del sistema de cara a su defensa y despliegue final.

[ADR-27] Resolución de Identidad mediante Claims del Token (Stateless Identity)

- **Fecha:** Junio 2026
- **Estatus:** Aceptado
- **Contexto:** El filtrado por cadenas de texto (**username**) presentaba fallos de integridad debido a la sensibilidad a mayúsculas de PostgreSQL ("Luis" vs "luis") y a la redundancia de consultas a la base de datos tras la validación del JWT.

- **Decisión:** Se ha migrado la resolución de identidad en los controladores hacia el uso del Claim `userId` transportado en el payload del token. El sistema extrae el ID numérico (clave primaria inmutable) directamente del contexto de `Authentication`, evitando llamadas extra a `findByUsername`.
 - **Justificación para el TFG:** Aporta solidez técnica al alinearse con las especificaciones de seguridad modernas de OAuth2 y JWT. En lugar de forzar al backend a realizar búsquedas repetitivas por cadenas de texto vulnerables a fallos de capitalización, el uso del ID extraído directamente de los claims asegura una resolución atómica y determinista.
 - **Consecuencias:**
 - **Eliminación de errores de capitalización:** La clave primaria numérica actúa como el único identificador unívoco e inmune a las variaciones de caracteres.
 - **Reducción de la latencia de red:** Se optimizan las conexiones con PostgreSQL al suprimir las consultas repetitivas de verificación de usuario.
 - **Cumplimiento estricto del patrón Stateless de REST:** El servidor no retiene estados de sesión, delegando de forma segura toda la identidad en el token cifrado.
-

[ADR-28] Hidratación Síncrona de Matrículas en el DTO de Sesión

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Tras aplicar `@JsonIgnore` en las relaciones de usuario para evitar recursiones infinitas y desbordamientos de memoria por bucles de serialización, el frontend perdió la capacidad de saber instantáneamente en qué cursos estaba inscrito el alumno tras iniciar sesión.
 - **Decisión:** Rediseñar el registro `AuthTokenResponse.java` para incluir de forma síncrona una lista de `enrolledCourseIds`. Este array se puebla durante el proceso de login mediante una consulta proyectada y optimizada en el repositorio.
 - **Justificación para el TFG:** Resuelve un problema clásico de asincronía y desacoplamiento en aplicaciones SPA. En lugar de obligar al frontend a disparar peticiones HTTP en cascada inmediatamente después del login para conocer el estado académico del alumno, el servidor inyecta proactivamente las referencias clave en la respuesta inicial de autenticación.
 - **Consecuencias:**
 - **Mejora drástica en la UX:** Los componentes del catálogo y los botones de inscripción reaccionan de manera instantánea sin pantallas de carga adicionales.
 - **Blindaje de serialización:** Se mantiene la protección contra la recursión infinita en las entidades JPA sin comprometer la entrega de datos esenciales al cliente.
-

[ADR-29] Estrategia de Carga Transaccional (Join Fetch vs OSIV)

- **Fecha:** Junio 2026
- **Estatus:** Aceptado

- **Contexto:** Con la directiva de configuración `spring.jpa.open-in-view=false` activa para evitar fugas de conexiones en el pool, el acceso a los detalles de un curso desde una matrícula disparaba errores 500 fuera de la capa de servicio debido a la naturaleza perezosa (*Lazy loading*) de Hibernate.
 - **Decisión:** Implementar consultas explícitas utilizando cláusulas `JOIN FETCH` en `EnrollmentRepository`. Esto fuerza al motor de persistencia a recuperar el curso asociado dentro del mismo ciclo transaccional de la base de datos.
 - **Justificación para el TFG:** Demuestra madurez de ingeniería en el manejo de ORMs y rendimiento de bases de datos relacionales. Evita caer en la mala práctica de habilitar OSIV (que mantiene hilos de conexión abiertos innecesariamente hacia PostgreSQL) y ataca directamente el problema desde la raíz del diseño de la consulta.
 - **Consecuencias:**
 - **Garantía de datos completos:** El frontend recibe el grafo de objetos hidratado de forma segura y libre de excepciones transaccionales.
 - **Optimización del rendimiento:** Se mitiga por completo el temido problema de las "N+1 consultas", unificando las lecturas en un único viaje a la base de datos.
-

[ADR-30] Algoritmo de Filtrado Basado en Contenido para el Motor de Recomendaciones

- **Fecha:** Junio 2026
 - **Estatus:** Aceptado
 - **Contexto:** Se requería un motor de sugerencias personalizado dentro del panel del estudiante que priorizara la afinidad temática, las competencias académicas y el historial de navegación previo del alumno en lugar de mostrar un catálogo estático.
 - **Decisión:** Desarrollar un servicio especializado (`RecommendationService`) que cruce los metadatos del catálogo (`Courses`) con el perfil de preferencias guardado (`Interest`) y las matrículas previas (`Enrollment`), aplicando una ponderación matemática de pesos (30% Categoría, 25% Historial, etc.).
 - **Justificación para el TFG:** Eleva el valor académico del proyecto al introducir una capa legítima de Inteligencia de Negocio y lógica algorítmica. Justifica técnicamente el diseño de la base de datos y la creación de las tablas satélite creadas mediante colecciones de elementos.
 - **Consecuencias:**
 - **Sugerencias predictivas dinámicas:** El frontend pinta en tiempo real recomendaciones justificadas con un porqué explícito para el estudiante.
 - **Aprovechamiento del modelo relacional:** Se saca el máximo partido a la normalización de metadatos (categorías, lenguajes, habilidades) estructurada en PostgreSQL.
-

[ADR-31] Estrategia de Carga de Preferencias: Robustez frente a Rendimiento en Colecciones

- **Fecha:** Junio 2026
- **Estatus:** Aceptado

Contexto

La entidad **Interest** mapea las colecciones de preferencias dinámicas del estudiante mediante la anotación **@ElementCollection**. Por defecto, Hibernate gestiona estas colecciones mediante una carga diferida (**FetchType.LAZY**). Sin embargo, debido a las múltiples transformaciones de DTOs en capas desacopladas fuera de la sesión transaccional y bajo el contexto de seguridad del filtro JWT, la persistencia se exponía a excepciones de tipo **LazyInitializationException** al cerrarse la sesión antes de la serialización JSON, lo que provocaba que el frontend recibiera listas vacías y disparara el banner de error rojo en la UI.

Decisión

Se determina implementar una **Estrategia de Hidratación Explícita Controlada** dentro del método de lectura transaccional **getUserInterests** de **UserService.java**. En lugar de forzar un acoplamiento estructural rígido mediante **@ElementCollection(fetch = FetchType.EAGER)** —el cual sobrecargaría el rendimiento con productos cartesianos inválidos en PostgreSQL—, se mantiene el esquema diferido eficiente (**LAZY**) y se ejecuta programáticamente el método de resolución de proxies invocando el tamaño de cada colección (**.size()**):

```
if (interest.getCategory() != null) interest.getCategory().size();
if (interest.getCourse_type() != null) interest.getCourse_type().size();
if (interest.getDuration() != null) interest.getDuration().size();
if (interest.getLanguage() != null) interest.getLanguage().size();
if (interest.getSubtitle_languages() != null)
interest.getSubtitle_languages().size();
```

Esto obliga al ORM a poblar las colecciones satélite de forma síncrona mientras la transacción **@Transactional(readOnly = true)** permanece abierta.

Justificación para el TFG

Refleja la capacidad del ingeniero para evaluar los compromisos de diseño (*trade-offs*) en la persistencia avanzada de datos. Ante el tribunal, se defiende como una decisión táctica, defensiva y limpia: se evita delegar la carga en intermediarios o inicializaciones globales pesadas del proveedor, controlando a nivel de servicio exactamente cuándo y cómo se resuelve el grafo de datos. Esto blindo la API contra fallos de proxy en entornos multihilo o filtros desacoplados, manteniendo la consistencia de tipos primitivos.

Consecuencias

- **Estabilidad absoluta de la API:** Se eliminan de raíz las excepciones de inicialización diferida al transformar los intereses del alumno a **InterestDTO**, garantizando el retorno seguro de los datos en un estado HTTP 200 OK.
- **Optimización de Recursos en PostgreSQL:** Al resolver los datos de forma dirigida en el Read Path del servicio, se previene el desperdicio de memoria en operaciones de escritura u otras consultas secundarias donde no se requiera el desglose multidimensional de intereses.
- **Sincronización Simétrica Documental:** Este registro interactúa y se hermana directamente con las directrices de persistencia del [ADR-18], cerrando de forma definitiva la coherencia técnica entre el código fuente transaccional de Spring Boot y la documentación del monorrepo.

[ADR-32] Algoritmo de Filtrado Basado en Contenido para el Motor de Recomendaciones

- **Fecha:** Junio 2026
- **Estatus:** Aceptado
- **Contexto:** Se requería un motor de sugerencias personalizado dentro del panel del estudiante que priorizara la afinidad temática, las competencias académicas y la disponibilidad de tiempo del alumno. El backend original carecía de lógica predictiva y el frontend dependía de datos estáticos (*mocks*), lo que reducía el valor tecnológico de la plataforma de cara a la defensa del proyecto.
- **Decisión:** Diseñar y desarrollar un servicio especializado (`RecommendationService.java`) que implementa un algoritmo de Filtrado Basado en Contenido (*Content-Based Filtering*). El motor opera de forma determinista bajo una matriz de pesos en memoria (30% Categoría, 25% Historial, 20% Nivel, 15% Idioma, 10% Duración). El enrutamiento se expone de forma segura en `/api/courses/recommendations` resolviendo la identidad mediante el Claim del token JWT y el cliente React se conecta de forma reactiva a través del gancho personalizado `useSmartRecommendations.ts`.
- **Justificación para el TFG:** Demuestra madurez de ingeniería al resolver el acoplamiento y la eficiencia del grafo de persistencia. En lugar de delegar el cálculo matemático de ponderación a PostgreSQL mediante costosos procedimientos almacenados o consultas complejas con subconsultas cíclicas, se adopta una **Estrategia Stateless**. Los datos se recuperan atómicamente mediante proyecciones nativas e indexadas de clave primaria (`findEnrolledCourseIdsByUserId`) y se procesan a alta velocidad en memoria utilizando *Java Streams*, liberando por completo de carga computacional al servidor de la base de datos.
- **Consecuencias:**
 - **Sugerencias predictivas dinámicas:** El frontend renderiza en tiempo real recomendaciones justificadas con un porqué explícito (`reason`) de forma transparente para el estudiante.
 - **Aislamiento e Inmunidad ante Regresiones:** El algoritmo queda blindado metodológicamente mediante una suite de pruebas de comportamiento automatizadas en JUnit 5 y Mockito (`RecommendationServiceTest`), certificando con un 100% de éxito en consola (`BUILD SUCCESS`) que el sistema descarta cursos irrelevantes y excluye de forma estricta las asignaturas en las que el alumno ya está matriculado.
 - **Arquitectura Altamente Reactiva:** La UI del frontend se sincroniza automáticamente e invalida el feed en milisegundos en cuanto el estudiante efectúa una nueva matrícula, disparando una recarga limpia de red sin necesidad de refrescar el navegador.

[Enmienda Tecnológica a ADR-33] Evolución de la Persistencia Multidimensional de Intereses

Contexto Técnico

Durante la integración del Motor de Recomendaciones Algorítmicas, se detectó un fallo de regresión en el guardado del modal de preferencias del estudiante. La tabla principal `interests` y sus 5 tablas satélite (ej. `interest_categories`) rechazaban las actualizaciones de forma silenciosa debido a dos fenómenos del ciclo de vida de Hibernate:

1. **Identidad Huérfana:** Al usar `@MapsId` sin estrategias `@GeneratedValue`, el motor de persistencia requería la asignación explícita del ID del usuario en memoria antes de invocar al repositorio.

2. **Violación de PersistentBag:** El uso de métodos *setter* convencionales reemplazaba los envoltorios nativos de las colecciones de Spring Data JPA por instancias estándar de `ArrayList`, inhabilitando la generación automática de sentencias SQL `DELETE` e `INSERT`.

Decisiones Adoptadas

1. **Sincronización Manual de Identidad:** Se modificó el bloque constructivo `.orElseGet()` en la capa de servicios para forzar la inyección del ID físico del usuario (`newInterest.setId(user.getUser_id())`) previo al volcado atómico.
2. **Patrón de Mutación Destructivo-Limpio [ADR-18]:** Se implementó una abstracción encapsulada mediante el método `updateCollection(current, next)`, aplicando operaciones estrictas de `.clear()` y `.addAll()` sobre las listas originales de Hibernate para preservar el rastreo del estado de persistencia (*dirty checking*).
3. **Tolerancia Semántica en Endpoints:** Se expandió la anotación del controlador a un mapa `@RequestMapping` multitolerante para absorber de forma nativa peticiones `POST` y `PUT` sin alterar el contrato con la interfaz de usuario en React.

Consecuencias Positivas

- Se restablece la suite de pruebas del frontend y backend en verde sin alterar las dependencias del `pom.xml`.
- Se garantiza la integridad referencial en PostgreSQL en la Primera Forma Normal (1FN), asegurando que el motor algorítmico lea datos reales e impidiendo el parpadeo de estado local en la interfaz del alumno Luis.

[ADR-34] Gestión de Sesión por Inactividad frente a Expiración Absoluta

Estado

Aceptado

Contexto

El Tiempo de Vida (TTL) del token JWT está configurado en 15 minutos (900 segundos) para mitigar ventanas de exposición ante posibles secuestros de sesión. En la implementación previa del frontend, el cliente HTTP calculaba un instante estático de caducidad absoluto (`Date.now() + 15min`) al procesar el inicio de sesión. Esta lógica provocaba la expulsión abrupta y prematura del alumno Luis desde el Dashboard, incluso si se encontraba redactando, interactuando de forma concurrente con el modal de intereses o consumiendo el catálogo de cursos. Adicionalmente, la auditoría del monorrepo arrojó un error en la fórmula matemática del "Clock Skew" del servidor y una doble identidad redundante en las claves de persistencia local (`user` y `auth_user`).

Decisión

1. **Activity Tracker en el Cliente:** Implementar un monitor de actividad reactivo dentro del `AuthProvider.tsx` que capture interacciones humanas nativas en el DOM (`mousemove`, `keydown`, `click`, `scroll`). Cada evento válido pospone el instante de vencimiento del usuario en memoria y en disco otorgándole un nuevo margen renovado de 15 minutos.
2. **Mecanismo de Throttle Embebido:** Limitar la frecuencia de actualización del estado de React a un rango de muestreo seguro de 2000 milisegundos mediante el uso táctico de una referencia inmutable (`useRef`). Esto bloquea re-renders masivos del Dashboard preservando el rendimiento gráfico de la interfaz.
3. **Corrección Aritmética del Clock Skew:** Modificar la validación temporal de `JwtService.java` en el servidor restando la tolerancia de 60 segundos del instante actual (`Instant.now().minusSeconds(skew)`) en lugar de sumarla, garantizando que el token disfrute de un margen de gracia real ante desincronizaciones de reloj entre cliente y servidor.
4. **Unificación Física del Storage:** Refactorizar `authStorage.ts` para persistir la sesión bajo una única constante e identificador físico inmutable (`auth_user`), mitigando el split-brain informático del `localStorage` y eliminando de un plumazo el consumo de cuota duplicado detectado en la auditoría.

Consecuencias

- **Mejora en Usabilidad:** Se elimina la expulsión prematura del alumno activo de la plataforma de manera transparente sin comprometer las directrices estrictas de seguridad.
- **Consistencia de Datos:** La suite de pruebas de Vitest se elevó a 33 de 33 tests en verde gracias al diseño de relojes virtuales (`vi.useFakeTimers`) que simulan interacciones concurrentes del usuario en el tiempo absoluto.
- **Aislamiento de Errores:** El servidor tolera desincronizaciones temporales de milisegundos en peticiones HTTP API REST complejas sin corromper el canal.

[ADR-35] Consistencia de Identidad Bidireccional y Ordenamiento Determinista en el Flujo de Matrículas

Estado

Aceptado

Contexto

Durante la fase de integración del *Activity Tracker*, el cálculo de progreso académico "al vuelo" (*on the fly*) de las asignaturas en curso introdujo una doble anomalía crítica en el flujo de trabajo del alumno:

1. **Desfase de Indexación Visual ("Efecto Dominó"):** Al accionar el evento del DOM "Iniciar curso" en la tarjeta superior de la lista vertical, la interfaz sufría un parpadeo visual y trasladaba erróneamente el estado activo ("✓ Estudiando asignatura") a la tarjeta inferior, dejando el nodo pulsado intacto.
2. **Falso Positivo de Expiración de Sesión:** Tras pulsaciones consecutivas sobre el componente reactivo, el sistema sufría una expulsión abrupta del usuario hacia la pantalla de login debido a la activación involuntaria del circuito de expiración de sesión definido en el `AuthProvider.tsx` [ADR-34].

La auditoría técnica reveló un quiebre de la integridad de identidad bidireccional entre la base de datos y el cliente asíncrono. En el backend, la consulta HQL de `EnrollmentRepository.java` carecía de una cláusula `ORDER BY` explícita, provocando que Hibernate alterara la disposición física de la fila en el cursor de PostgreSQL tras ejecutar el `UPDATE` de la estampa de tiempo `started_at`.

En el frontend, una discrepancia nominal en el hook `useEnrolledCourses.ts` provocaba que la clave primaria de la matrícula se mapeara como `undefined`. Ante la ausencia de un identificador estable, el algoritmo de reconciliación de React (*diffing algorithm*) recurría de forma implícita al índice relativo del array (`key={index}`). Al mutar los datos en el backend y reordenarse el array devuelto, React asociaba el nuevo estado visual a la posición indexada previa y no a la entidad real.

Finalmente, el valor `undefined` se propagaba a la pasarela HTTP (Axios), mutando la URL del endpoint a una ruta malformada (`/api/auth/enrollment/undefined/start`). El componente `JwtAuthenticationFilter.java` de Spring Security rechazaba la petición por anomalía de ruta respondiendo con un `HTTP 401 Unauthorized`. El interceptor global de red capturaba el 401, interpretaba falsamente que el token JWT había caducado y disparaba el evento `auth-session-expired`, expulsando al usuario del sistema.

Decisión

- 1. Ordenamiento Determinista en Persistencia:** Modificar la consulta HQL del método `findAllByUserIdWithCourses` en `EnrollmentRepository.java` inyectando una cláusula estricta de ordenación: `ORDER BY e.enrollmentid ASC`. Esto ancla de forma invariable el cursor físico de PostgreSQL ante escrituras concurrentes, manteniendo el mecanismo de optimización `JOIN FETCH` para neutralizar excepciones de inicialización diferida (*LazyInitializationException*).
- 2. Normalización Estricta del Contrato DTO:** Refactorizar el mapeo síncrono del hook `useEnrolledCourses.ts` para capturar la clave primaria exacta (`enrollmentid` en minúsculas) generada por el ORM. Se prohíben de forma taxativa los mecanismos de "salvavidas" o *fallbacks* inestables (como IDs de cursos o marcas de tiempo arbitrarias) que diluyan la identidad de la matrícula. Se depuran además las importaciones muertas (`DBModelCourse`) para satisfacer las reglas del linter de TypeScript.
- 3. Implementación de Semáforo de Mutación (UI Lock):** Introducir un estado local de control de concurrencia (`mutatingId`) en el componente `EnrolledCourses.tsx`. Este mecanismo bloquea y deshabilita los controles de la tarjeta en proceso de activación, impidiendo ráfagas de clics repetidos provocadas por el parpadeo de red y proporcionando un indicador visual asíncrono (*spinner*) de carga.
- 4. Evitación de Atributos Nativos del DOM:** Sustituir la extracción de identificadores a través de atributos nativos del navegador (`e.currentTarget.getAttribute`) por pasajes de parámetros tipados directos en el ámbito de la función de React, eliminando fallos de parseo en caliente.
- 5. Cumplimiento de la Política de Estilos Estáticos:** Eludir la regla estricta de análisis de Microsoft Edge Tools (`no-inline-styles`) mediante el uso de la sintaxis de propagación de objetos de JavaScript (`{...{ style: { width: ... } }}`) sobre la barra de progreso de Tailwind, preservando la animación reactiva y la limpieza del panel de problemas de VS Code.

Consecuencias

- Estabilidad del DOM Virtual:** Las identidades de las tarjetas en curso quedan fijadas de manera inmutable a su clave primaria real de PostgreSQL. El algoritmo de reconciliación de React actualiza los

nodos físicos de forma exacta, eliminando el "efecto dominó" visual y garantizando la predictibilidad de la interfaz de usuario bajo transiciones CSS de Tailwind.

- **Robustez en la Capa de Seguridad:** Al garantizar la integridad del `enrollmentid` en el frontend, se erradican por completo las peticiones dirigidas a rutas corruptas. El filtro de Spring Security procesa exclusivamente URLs válidas bajo el token JWT activo, deteniendo las expulsiones forzadas involuntarias en el flujo de interacción.
- **Mantenimiento de la Cobertura de Pruebas:** Las refactorizaciones aplicadas preservan el comportamiento esperado por la suite de pruebas del monorrepo. Tras la unificación del tipado en las propiedades del componente (`EnrolledCoursesProps`), los 35 de 35 tests unitarios e integrados en Vitest (`EnrolledCourses.test.tsx`) se mantienen estables en verde.

[ADR-36] Módulo de Intercambio Bidireccional y Dirigido de Documentos Académicos

Estado

Aceptado

Contexto

Para potenciar la interacción pedagógica en la plataforma, surgió la necesidad de transformar el módulo original de almacenamiento plano y unidireccional en un sistema de intercambio documental bidireccional y dirigido. La directriz original del [ADR-23] limitaba la carpeta de documentos de forma estricta a extensiones `.pdf` para neutralizar vectores de ataque por ejecución remota de código (RCE) y suplantación de tipos (*MIME-sniffing*).

Permitir la incorporación de formatos de procesamiento de palabras como Microsoft Word (`.docx`) y texto plano (`.txt`) requería una reevaluación del perímetro de seguridad para evitar que la flexibilización de extensiones comprometiera la integridad del servidor. Asimismo, el diseño de negocio exigía que el intercambio ocurriera estrictamente entre personas físicas registradas en la plataforma, utilizando la entidad de cursos (`Courses.java`) no como almacén, sino como un cortocircuito de seguridad perimetral para validar la red académica legítima (profesores matriculados, compañeros de clase y administradores globales), aislando la lógica de red para evitar la proliferación de componentes sobredimensionados (*God Components*) [ADR-20].

Decisión

1. **Ampliación Perimetral Dual:** Modificar `FileStorageService.java` para validar de forma síncrona y obligatoria que el archivo transmitido satisfaga simultáneamente la extensión de disco permitida (`pdf`, `docx`, `txt`) y su respectivo tipo MIME oficial (`application/pdf`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`, `text/plain`). Cualquier payload que quiebre una de las dos condiciones es rechazado inmediatamente (HTTP 400).
2. **Modelo Dirigido Emisor-Receptor (Contrato de Mensajería entre Usuarios):** Descartar la propiedad simple acoplada a un único actor y evolucionar la entidad `DocumentMetadata.java` hacia un contrato de mensajería dirigido. La tabla registra claves foráneas (`sender_id` y `receiver_id`) que apuntan de forma exclusiva a identificadores reales de la entidad `Users.java`. Se prohíbe el autoenvío mediante

una restricción **CHECK** en PostgreSQL. Las bandejas de visualización en el frontend se aíslan de forma física en base de datos mediante el enumerado **FolderType** (**SENT** y **RECEIVED**).

3. **Validación de Ámbito Académico y Directorio Dinámico:** Implementar en el Read Path del backend endpoints analíticos de consulta en **UserRepository.java** y **EnrollmentRepository.java**. Estos endpoints cruzan en tiempo real el historial de matrículas activas del alumno con el campo estático **instructors** de la entidad de cursos para aislar su red legítima de contactos, devolviendo únicamente objetos de transferencia de datos limpios (**UserDirectoryDTO**) que protegen el hash de las contraseñas e inyectan el identificador obligatorio del destinatario.
4. **Bifurcación en Servicios y Hooks de Presentación:** Actualizar **documentService.ts** para integrar de forma asíncrona los métodos de bandeja de entrada, bandeja de salida y consultas al directorio. El hook especializado **useDocuments.ts** asume la gestión mutada de estados independientes de las listas de enviados y recibidos para evitar el "efecto fantasma" visual [ADR-19], controlando de forma reactiva la pestaña activa (**activeTab**) y sanitizando excepciones de tipo **unknown**.
5. **Composición Gráfica por Alturas Proporcionales Compensadas:** Rediseñar el componente **DocumentManager.tsx** bajo criterios de composición pura [ADR-13], delimitado a una tarjeta geométrica de **h-109**. Se introduce un sistema de pestañas interactivas, un selector obligatorio **<select>** blindado contra nulos y dotado del atributo **aria-label**. Para garantizar la simetría e integridad visual en el Dashboard general del alumno Luis, se establece una **Estrategia de Columnas Equivalentes**: la columna izquierda apila el catálogo/asignaturas junto a la gestión de documentos (**h-109**), mientras que la columna derecha equilibra la interfaz mediante la tarjeta de **EvaluationPanel.tsx** fijada intencionadamente en **h-90**. Esto absorbe las diferencias de densidad de información y consolida una línea inferior de maquetación perfectamente nivelada.

Consecuencias

- **Seguridad, Integridad y Eficiencia:** El servidor acepta múltiples formatos de texto de forma segura sin peligro de ataques de sobreescritura de archivos (*Path Traversal*) gracias al enmascaramiento con UUID en disco. El modelo de datos garantiza que el intercambio ocurre únicamente entre usuarios reales registrados en PostgreSQL. Se introduce un cortocircuito defensivo de negocio en el controlador que frena peticiones inválidas antes de consumir recursos de infraestructura.
- **Estabilización de la Simetría Visual:** Se corrige la inconsistencia teórica de la maquetación en la documentación. El ajuste justifica formalmente ante el tribunal que la coexistencia de clases de dimensiones diferenciadas (**h-109** en documentos y **h-90** en evaluación) no es un descuido de diseño, sino un cálculo preciso de distribución proporcional para mantener la cuadratura perfecta del layout inferior.
- **Mantenimiento y Cobertura:** Se mantiene la política rigurosa de calidad de código del monorrepo. La capa de presentación frontend se validó mediante el desarrollo de **DocumentManager.test.tsx** en Vitest, garantizando la cobertura de los estados vacíos contextuales por pestañas y el bloqueo perimetral del input de subida si no hay destinatario. La capa de control backend se blindó mediante pruebas unitarias puras de aislamiento con Mockito en **DocumentControllerTest.java**, asegurando que las subidas dupliquen correctamente los registros para emisor y receptor de forma síncrona. Ambas suites se ejecutan con un éxito del 100% y se consolidan en verde.

[ADR-37] Sistema de Evaluación Académica y Arquitectura de Rating Dual

Estado

Aceptado

Contexto

Para potenciar los criterios de madurez pedagógica de la plataforma, se requería un módulo que permitiera a los estudiantes calificar tanto la calidad del material de la asignatura como el desempeño del profesorado. No obstante, el dataset original extraído de Coursera presentaba dos restricciones estructurales:

1. Los instructores no poseían entidades relacionales propias en PostgreSQL, sino que se almacenaban como cadenas de texto indexadas (**TEXT**) dentro de la tabla de cursos.
2. Los cursos poseían una puntuación estática original (**rating**). Alterar directamente dicha columna con las notas locales de la plataforma introduciría una alta volatilidad estadística en las medias aritméticas debido a la asimetría del tamaño muestral (N inicial masivo del dataset frente a interacciones locales).

Adicionalmente, bajo las políticas de seguridad perimetral por tokens distribuidos [ADR-27][ADR-35], era obligatorio restringir el formulario únicamente a aquellos docentes e itinerarios formativos en los que el alumno contara con una matrícula vigente y que no hubieran sido evaluados previamente, mitigando ráfagas de fraude o duplicación de votos.

Decisión

1. **Arquitectura de Rating Dual:** Implementar un patrón de coexistencia analítica. La interfaz del frontend consumirá de manera diferenciada el "Rating del Catálogo" (proveniente de las columnas originales del dataset) y el "Rating de la Comunidad" (calculado dinámicamente en tiempo de ejecución mediante funciones de agregación **AVG** sobre los registros locales de la plataforma), preservando la pureza de la fuente original de datos.
2. **Granularidad Disociada y Envío Asimétrico:** Diseñar la entidad **AcademicEvaluation.java** segregando las puntuaciones y comentarios en campos independientes. Se refactoriza la condición lógica de la UI (**canSubmit**) para habilitar el envío asimétrico del formulario. El estudiante tiene plena libertad para calificar exclusivamente la calidad del curso, únicamente el desempeño docente, o ambos bloques en paralelo, exigiendo como único requisito de negocio que al menos una de las dos métricas sea mayor que cero para mitigar payloads vacíos.
3. **Validación Perimetral por Subconsulta Cruzada:** Delegar el filtrado de exclusión a una subconsulta nativa en HQL dentro de **EnrollmentRepository.java**. El método descarta en el motor de PostgreSQL cualquier curso que ya posea una fila de evaluación acoplada al **username** extraído de los Claims del JWT, blindando el acceso en el endpoint **/pending** de forma stateless.
4. **Accesibilidad e Integridad Sintáctica en la UI:** Refactorizar el renderizado iterativo del panel de estrellas inyectando de forma obligatoria propiedades descriptivas **aria-label** y **title** para satisfacer los inspectores automatizados de accesibilidad (*Microsoft Edge Tools / axe*). La estructura se mantiene desacoplada en el hook **useActiveEvaluations.ts** satisfaciendo la directriz anti-objetos mutantes del linter [ADR-20].

Consecuencias

- **Consistencia Analítica:** Se habilita un sistema de puntuación docente y de asignaturas altamente descriptivo sin degradar el dataset preprocesado en Python. Las consultas agregadas computan el

String del docente de forma unívoca, resolviendo la carencia de identificadores físicos de instructores en la base de datos.

- **Validación de Reglas de Negocio:** El controlador bloquea de forma autónoma (HTTP 403 Forbidden / HTTP 400 Bad Request) cualquier intento de inyección de payloads corruptos o evaluaciones duplicadas, protegiendo las tablas relacionales.
- **Robustez en la Suite de Integración:** La suite de pruebas de interfaz del cliente se eleva a un total consolidado de 46 tests en verde. La integración de bloques asíncronos `await` `waitFor` dentro de `EvaluationPanel.test.tsx` elimina con éxito las alertas de fugas de estado transitorias de React Testing Library, garantizando la predictibilidad de la plataforma ante futuras auditorías de código.

[ADR-38] Descarga Segura de Documentos Académicos y Control de Acceso Anti-IDOR

Estado

Aceptado

Contexto

Para consolidar las directrices de privacidad y control de acceso en el módulo de intercambio bidireccional y dirigido de archivos, se requería una solución técnica que garantizara que los documentos académicos almacenados en el servidor solo pudieran ser descargados por sus destinatarios legítimos o sus emisores originales. El diseño arquitectónico inicial presentaba tres vulnerabilidades críticas:

1. La ruta estática `/uploads/**` estaba configurada como pública en la seguridad perimetral, permitiendo que cualquier usuario malintencionado que conociera el nombre físico del archivo pudiera saltarse los filtros de autenticación y descargarlo de forma directa.
2. El uso de `window.open(fileUrl)` en el frontend exponía metadatos e identificadores sensibles en la barra de direcciones del navegador, facilitando vectores de ataque basados en la enumeración predecible de recursos.
3. El sistema carecía de un mecanismo de validación de identidad en tiempo de ejecución, lo que exponía la plataforma a vulnerabilidades de Referencia Directa Insegura a Objetos (IDOR), donde un alumno autenticado válidamente con su token JWT podía consultar recursos privados pertenecientes a otros alumnos modificando los parámetros de la solicitud.

Decisión

1. **Aislamiento Perimetral y Blindaje de Archivos:** Eliminar por completo la regla estática `.requestMatchers("/uploads/**").permitAll()` en `SecurityConfig.java`. El directorio de almacenamiento físico queda completamente aislado del tráfico de red externo, obligando a que cualquier solicitud de lectura sea interceptada y evaluada por el contexto de Spring Security.
2. **Cortocircuito Defensivo Anti-IDOR (Backend):** Implementar el endpoint protegido `GET /api/v1/documents/download/{documentId}` en `DocumentController.java`. El método recupera el token JWT a través del objeto `Principal`, intercepta la entidad en la base de datos de PostgreSQL y evalúa mediante una condición excluyente si el `username` del usuario autenticado coincide obligatoriamente con el emisor (`sender_id`) o el receptor (`receiver_id`) del documento. Si la

validación falla, el flujo se interrumpe de forma reactiva respondiendo con un estado HTTP 403 Forbidden.

3. **Consumo por Flujo de Datos Binarios (Frontend):** Sustituir el uso de `window.open` en el cliente por un consumo asíncrono basado en `Blob`. La función `downloadDocumentSecure` inyecta de forma transparente el token JWT en las cabeceras de autorización mediante el cliente HTTP de Axios (`ApiClient`) y procesa el flujo de bytes directamente en la memoria del navegador, forzando la descarga local con el nombre real del archivo (`originalname`).
4. **Control de Concurrencia y Estado Visual en la UI:** Refactorizar el componente `DocumentManager.tsx` inyectando un estado de bloqueo local denominado `downloadingId`. Al activar el evento `onClick`, el componente inhabilita el botón correspondiente (`disabled`) para evitar solicitudes HTTP simultáneas o dobles clics accidentales del usuario. Simultáneamente, se muta el icono estático por un spinner animado (`Loader2`) preservando las dimensiones geométricas y las clases de Tailwind CSS sin alterar la maqueta.

Consecuencias

- **Mitigación Completa de Vulnerabilidades IDOR:** Se erradica la posibilidad de fugas de información por manipulación de identificadores en la URL. El servidor valida la matriz de permisos de forma interna, garantizando la confidencialidad estricta del material académico.
- **Flujos de Datos Transparentes:** Las credenciales de autorización y los tokens JWT viajan encapsulados de forma oculta en las cabeceras HTTP, evitando la exposición de firmas digitales en el historial del navegador o en los logs del servidor proxy.
- **Integridad de la Suite de Pruebas:** Los tests de integración con `MockMvc` en `DocumentControllerTest.java` validan con éxito que los intentos de intrusión devuelvan un error 403. En el cliente, la refactorización mantiene en verde un consolidado absoluto de 46 tests en Vitest, garantizando que el comportamiento asíncrono y los bloqueos de interfaz se ejecutan de manera predecible y libre de regresiones.

[ADR-39] Diseño de Contrato Anticipado para la Integración Desacoplada de Calificaciones Académicas

Estado

Aceptado

Contexto

Con el objetivo de expandir las capacidades formativas del panel del estudiante (`EnrolledCourses.tsx`) hacia futuras interacciones con el módulo del profesorado, se requería habilitar una "ventana informativa de calificaciones" que permitiera reflejar el motivo (exámenes, trabajos prácticos) y la nota asignada.

No obstante, debido a que el módulo de inserción de calificaciones por parte del docente se implementará en una fase posterior del desarrollo, la arquitectura del frontend requería un diseño que cumpliera con las siguientes restricciones estructurales:

1. **Tipado Estricto de Datos:** Evitar el uso de tipos genéricos (**any**) que degradaran la robustez del linter y la predictibilidad del compilador de TypeScript.
2. **Funcionalidad Inerte Conectable:** El componente debía ser visualmente operativo y tolerante a la ausencia de datos en el presente, pero quedar "arquitectónicamente listo" para reaccionar de forma automática en cuanto el backend comience a proveer los registros reales de PostgreSQL.
3. **Consistencia Tipográfica y Minimalismo:** La UI de calificaciones debía mimetizarse con el entorno visual del Dashboard del estudiante Luis, evitando sobrecargas cognitivas, redundancia de iconos o desalineaciones en la tarjeta contenedora rígida (**h-109**).

Decisión

1. **Estrategia de Pre-Hidratación y Contrato por Desacoplamiento:** Extender el contrato de transferencia de datos en **courseTypes.ts** mediante la creación de la interfaz **CourseGradeInfo** y su acoplamiento como propiedad opcional (**grades?**) dentro de **EnrollmentInfo**. Esto permite al cliente tipar de manera estricta el flujo de datos sin obligar al backend a enviar el payload en la fase actual.
2. **Evaluación de Ausencia Dinámica en el JSX:** Inyectar una sección condicional dentro del bucle **.map** que evalúe de forma reactiva la existencia de calificaciones. Si la colección es nula o vacía, la interfaz se comporta de manera inerte renderizando un estado elegante por defecto ("*Aún no hay calificaciones publicadas por el docente*"), eliminando cualquier riesgo de quiebre del ciclo de renderizado.
3. **Unificación Estética de Carga Gráfica:** Ajustar la coloración de la etiqueta del título a un tono gris sólido (**text-slate-700**) y prescindir de iconografía adicional (**GraduationCap**). Esto garantiza una integración cromática perfectamente simétrica con las etiquetas de categoría y textos perimetrales ya existentes en la plataforma.

Consecuencias

- **Desacoplamiento Efectivo de Módulos:** El equipo de frontend y backend pueden trabajar de forma asíncrona. La vista del estudiante queda blindada ante futuras modificaciones; cuando el controlador del profesor guarde datos en las tablas relacionales, la UI del alumno los consumirá nativamente sin necesidad de alterar una sola línea de código en la capa de presentación.
- **Mantenimiento del Espacio Geométrico:** El diseño compacto aprovecha de forma óptima los píxeles recuperados en la cabecera en el [ADR-38], manteniendo a salvo el scroll controlado de la tarjeta y la simetría visual con los componentes adyacentes del catálogo.
- **Cero Regresiones en la Suite de QA:** La suite de pruebas de **Vitest se consolida exitosamente con sus 46 tests en verde (PASS)**, demostrando que la adición de la lógica condicional y las interfaces tipadas no introducen comportamientos erráticos o fugas de memoria transitorias en React.

[ADR-40] Persistencia Relacional e Hidratación de Calificaciones con Aislamiento Deserializador READ_ONLY

Estado

Aceptado

Contexto

Dando continuidad al diseño de contrato anticipado establecido en el [ADR-39] para el módulo de calificaciones académicas, se requería materializar la estructura física de datos en el backend (Spring Boot + PostgreSQL + Hibernate). El reto residía en habilitar el almacenamiento dinámico de múltiples notas por cada matrícula sin vulnerar los principios de seguridad perimetral de la plataforma.

Específicamente, se debían mitigar dos vectores de riesgo críticos:

1. **Ataques de Asignación Masiva (Mass Assignment):** Impedir que un usuario malintencionado intente inyectar o alterar calificaciones enviando colecciones manipuladas en los payloads de solicitudes mutantes (`POST/PUT`).
2. **Degradación de Rendimiento (N+1 Query Problem):** Evitar que la consulta recurrente de matrículas sobrecargue la base de datos PostgreSQL al traer de manera ansiosa (*Eager*) colecciones de notas cuando no son requeridas por el contexto de la vista.

Decisión

1. **Modelado Físico Normalizado y Bidireccional:** Crear la entidad `CourseGrade.java` vinculada mediante una relación de muchos a uno (`@ManyToOne`) con la matrícula (`Enrollment`). La entidad principal `Enrollment.java` incorpora la contraparte inversa (`@OneToMany`) con estrategias de cascada completa (`CascadeType.ALL`) y remoción de huérfanos para garantizar la integridad referencial en cascada ante limpiezas de historial.
2. **Aislamiento de Deserialización Perimetral:** Sustituir la anotación restrictiva `@JsonIgnore` por `@JsonProperty(access = JsonProperty.Access.READ_ONLY)` sobre la propiedad `grades`. Esto instruye al serializador Jackson a ignorar de forma proactiva cualquier entrada de datos hacia el servidor a través de esta propiedad, utilizándola exclusivamente como flujo de salida seguro e inmutable hacia el Frontend.
3. **Optimización por Carga Diferida (Lazy Loading) e Hidratación Explícita:** Configurar la relación con `FetchType.LAZY` para salvaguardar el rendimiento del motor de base de datos. En el método transaccional de lectura `getStudentActiveCoursesWithCalculatedProgress` dentro de `UserService.java`, se fuerza la hidratación controlada de la colección invocando un método de acceso estructural (`.size()`) antes del retorno de la lista, poblando el payload únicamente en este flujo de negocio específico.
4. **Restauración de Firma de Constructores:** Declarar un constructor explícito de 7 argumentos en `Enrollment.java` que actúe como puente de compatibilidad hacia atrás. Esto inmuniza a la suite preexistente de pruebas unitarias (`UserServiceTest.java`, etc.) contra los cambios en las anotaciones automáticas de Lombok.

Consecuencias

- **Seguridad por Diseño (Security by Design):** Se blindó el endpoint contra inyecciones fraudulentas de notas desde el cliente. Las calificaciones quedan completamente aisladas en modo de solo lectura desde la perspectiva de la API pública expuesta.
- **Tolerancia Dual Frontend-Backend:** El backend ahora sirve un arreglo `grades: []` vacío pero estructuralmente válido. React asimila este payload de forma nativa manteniendo el comportamiento inerte actual y reaccionará de manera automatizada en el momento en que se inserten filas reales en la tabla `course_grades`.

- **Estabilidad del Entorno de Integración:** Tanto el compilador de Java 17 como los tests de JUnit/Mockito mantienen un estado 100% libre de errores y regresiones, preservando la madurez tecnológica de la plataforma de cara a futuras auditorías del tribunal.

[ADR-41] Arquitectura de Agregación Analítica Inmutable y Casteo Dinámico para Métricas de Catálogo

Estado

Pospuesto

Contexto

Para potenciar la madurez pedagógica de la plataforma, se diseñó la incorporación de un nuevo componente estadístico en el Dashboard del estudiante que consolidara métricas clave en tiempo real: nota media del curso, volumen local de inscritos y valoraciones medias disociadas de la asignatura y del profesorado. Asimismo, el componente debía estructurarse de forma extensible para admitir futuras segmentaciones analíticas basadas en atributos intrínsecos de la entidad de cursos (diferentes plataformas de origen y categorías temáticas).

No obstante, la implementación de este módulo analítico cruzado presentaba tres restricciones estructurales críticas:

1. **Incompatibilidad de Tipos en PostgreSQL:** El campo `score` de la entidad `CourseGrade` se definió originalmente como `String` para dotar de flexibilidad al sistema [ADR-39]. Intentar aplicar una función de agregación estándar `AVG()` sobre una columna de texto provocaría una excepción fatal inmediata en el motor de base de datos.
2. **Riesgo de Inyección de Payload:** Evitar vulnerabilidades de alteración masiva de datos (*Mass Assignment*), garantizando que las métricas de salida computadas por el servidor no puedan ser manipuladas o inyectadas artificialmente por peticiones maliciosas procedentes del cliente.
3. **Rendimiento e Integridad de Consultas:** Prevenir el problema de consultas masivas (N+1) o productos cartesianos inválidos al cruzar tablas de alta volatilidad analítica (`Enrollment`, `CourseGrade`, `AcademicEvaluation` y `Courses`).

Nota de Auditoría de Desarrollo: Tras la estabilización del núcleo transaccional del catálogo predictivo y el sistema de intereses del estudiante, se detectó la necesidad de priorizar el desacoplamiento de estas funciones básicas antes de acoplar de forma reactiva el panel analítico. Para mitigar condiciones de carrera y asegurar la consistencia del despliegue actual, se decide pausar temporalmente la activación de este módulo.

Decisión

Se determina **posponer e inactivar temporalmente la integración del componente analítico** en la fase de desarrollo actual, manteniendo el diseño de su arquitectura técnica pre-establecido en estado inerte para su futura reactivación coordinada con la página del profesor. Las directrices de diseño planificadas y validadas de forma teórica constan de:

1. **Casteo Dinámico Condicional en JPQL:** Planificación de una directiva de formateo al vuelo dentro de `CoursesRepository.java` utilizando la función nativa `CAST(cg.score AS double)`. Para inmunizar al motor de PostgreSQL contra errores de formato ante cadenas vacías o no numéricas, el casteo se encapsula dentro de una expresión condicional `CASE WHEN` que evalúa la presencia legítima de caracteres válidos antes de computar la media aritmética (`AVG`).
2. **Mapeo de Proyección Directa mediante Records:** Configuración de la estructura de transferencia de datos `CourseStatsDTO.java` haciendo uso de un `Record` inmutable de Java 17. La consulta del repositorio instanciará el DTO directamente desde las tuplas de PostgreSQL mediante la sintaxis `SELECT new ...`, evitando la sobrecarga en memoria que implicaría la hidratación de entidades JPA pesadas.
3. **Aislamiento Perimetral Analítico:** Restricción de todas las propiedades del `Record` con la anotación `@JsonProperty(access = JsonProperty.Access.READ_ONLY)`. Esto establece un blindaje de solo lectura a nivel de API, permitiendo la serialización limpia de las métricas hacia React pero desestimando cualquier intento de manipulación o inserción externa.
4. **Controlador Analítico Desacoplado:** Centralización de la exposición del servicio en `CourseStatsController.java` bajo la ruta `/api/v1/stats`. El endpoint validará de forma implícita la identidad del alumno mediante el contexto `Authentication` provisto por los Claims firmados del token JWT [ADR-27], neutralizando vectores de ataque por enumeración de recursos.

Consecuencias

- **Preservación de la Estabilidad:** La inactivación de este módulo analítico reduce la complejidad transaccional del proyecto actual, garantizando que el catálogo predictivo y el registro de intereses funcionen con total fluidez y con cero errores de red en la UI.
- **Garantía de Documentación Metodológica:** Mantener el ADR bajo el estado de *Pospuesto* justifica de manera académica ante el tribunal que el problema de agregación analítica de datos mixtos (String a Double en PostgreSQL) fue analizado, modelado y resuelto teóricamente, aportando valor al diseño de arquitectura del TFG.
- **Consistencia del Repositorio Actualizado:** Al retirar los disparadores reactivos del frontend y las consultas complejas del backend, la suite de pruebas y la compilación (`BUILD SUCCESS`) regresan a un terreno de consistencia absoluta, libre de regresiones colaterales en el ecosistema transaccional.

ADR-42: Tipado Defensivo y Gestión de Precisión en el Progreso Académico

Estado

Aceptado

Contexto

El motor de preprocesamiento de datos (Python) identifica que el catálogo de cursos contiene duraciones con valores decimales flotantes (ej. 2652.8 horas). Sin embargo, el método de cálculo de progreso académico en el backend (`calculateCurrentProgress`) realiza un truncamiento explícito a tipo primitivo `Long` mediante `.longValue()`. Es necesario justificar formalmente esta pérdida de precisión frente al riesgo de desborde o inconsistencia en la interfaz de usuario.

Decisión

Se decide mantener el uso del tipo `Long` para almacenar y procesar el total de horas de un curso dentro de la lógica de negocio del backend. Esta decisión se fundamenta en:

- **Mitigación de Errores Flotantes:** Evitar el uso de `Double` o `Float` en operaciones aritméticas de porcentaje previene que residuos decimales infinitos provoquen desajustes de redondeo en el árbol de renderizado de React.
- **Absorción por Redondeo de UI:** Dado que el método aplica un truncamiento final hacia abajo (`Math.floor`) y acota el resultado estrictamente a un entero (`int`) entre 0 y 100, la desviación máxima de precisión provocada por el truncamiento a `Long` es aritméticamente insignificante (inferior al 0.03% en cursos masivos) e invisible para el estudiante.

Consecuencias

- **Positivas:** Estabilidad absoluta en el tipo de retorno enviado a la API. Se eliminan comportamientos erráticos (como barras de progreso que superen el 100% o que muestren decimales infinitos en la maqueta visual).
- **Negativas:** Existe un desfase teórico de minutos en el cómputo total de cursos masivos, el cual es absorbido intencionadamente por el diseño de la experiencia de usuario.

ADR-43: Abstracción de Infraestructura y Purificación Semántica de la Interfaz (UX)

Estado

Aceptado

Contexto

Las primeras versiones de la interfaz del estudiante exponían términos técnicos explícitos de la infraestructura backend (ej. "*Sincronizado con PostgreSQL*") y mostraban la duración de las asignaturas en formato bruto de horas. Esto violaba el principio de ocultación de información y reducía la legibilidad académica del producto.

Decisión

Se establece un principio estricto de **Purificación Semántica y Abstracción de Capas**. Queda prohibido filtrar terminología de la persistencia o de la base de datos hacia la capa de presentación del alumno. Asimismo, se transforma de forma nativa la visualización del tiempo de estudio:

- La duración se calcula en el cliente transformando las horas a **Días Académicos** mediante un redondeo defensivo hacia arriba (`Math.ceil(duration / 24)`).

Consecuencias

- **Positivas:** Mejora drástica de la experiencia de usuario (UX). La interfaz se orienta puramente al negocio educativo, facilitando la planificación temporal real del estudiante Luis sin exponer detalles del stack tecnológico.
- **Negativas:** Requiere un esfuerzo adicional de formateo en los componentes frontend del catálogo y el dashboard para asegurar la conversión homogénea de unidades.

[ADR-44] Arquitectura de Agregación Analítica Disociada y Micro-indicadores

Fecha: Julio 2026

Estatus: Aceptado

Contexto

Se requiere proyectar métricas en tiempo real que combinen el catálogo base de cursos externos (origen, especialidad) con la analítica dinámica de los usuarios en la plataforma local (inscripciones, notas, valoraciones de docentes).

Los principales desafíos son:

1. **Heterogeneidad de Datos:** Coexistencia de metadatos estáticos de plataformas externas con agregaciones SQL numéricas mutables ejecutadas en PostgreSQL.
2. **Eficiencia y Carga de Memoria:** Riesgo de saturar el motor de base de datos con consultas costosas de agregación sobre entidades pesadas JPA.
3. **Manejo de Nulidad Pedagógica:** Gestión defensiva en la UI para representar asignaturas sin actividad o registros iniciales sin desconfigurar la maqueta.

Decisión

1. **Agregación en Proyección (Spring Boot):** Utilizar consultas nativas o JPQL en el repositorio que mapeen los resultados agregados (`COUNT`, `AVG`) directamente hacia un `CourseStatsDTO`, evitando instanciar entidades completas en memoria.
2. **Casteo Condicional y Coalesce:** Implementar lógica de control en el backend para gestionar la nulidad pedagógica (ej. retornar `---` en lugar de `NaN` o `0` cuando no existan calificaciones registradas).
3. **UI de Doble Dimensión Sincronizada (React + Tailwind):** Descartar la unificación masiva de datos en una sola cuadrícula. En su lugar, el componente `StudentStatsPanel` encapsula un contenedor unificado de altura controlada (`GenericCard`) subdividido en dos sub-cajas verticales totalmente independientes:
 - **Caja A ("Métricas de tu Campus"):** Dedicada a los datos dinámicos locales (Comunidad, Nota Media, Rating de Curso y Rating Docente).
 - **Caja B ("Ficha Técnica del Curso"):** Dedicada a los metadatos de integración del catálogo base (Origen Remoto y Especialidad).
4. **Resiliencia Visual:** Ambas sub-cajas operan con scrolling vertical inteligente (`overflow-y-auto`). Esto garantiza que la interfaz mantenga su simetría física actual intacta y permanezca libre de barras de scroll espurias, pero responda de forma adaptativa y automática si la entidad `Course` incrementa sus atributos en el futuro.

Consecuencias

- **Positivas:** Reducción drástica del overhead de red, renderizado reactivo sin basura visual, interfaz de usuario desacoplada y preparada para futuras extensiones del catálogo sin comprometer la consistencia geométrica del Dashboard.
- **Negativas:** Obliga a mantener una estricta correspondencia entre las alturas relativas de las sub-cajas dentro del contenedor padre para evitar solapamientos visuales.

Notas de Migración: Transición a JWT y Compatibilidad

Fecha de análisis: Junio 2026 **Objetivo:** Definir el contrato de autenticación, compatibilidad y alcance de la transición desde el modelo de sesiones actual hacia JSON Web Tokens (JWT).

1. Lógica de Sesiones Actual (A mantener temporalmente)

- **Base en Spring Security:** Configuración activa en `SecurityConfig.java` con política `IF_REQUIRED`. El `SecurityContext` se persiste en la sesión HTTP tradicional.
 - **Flujo de Login:** Gestionado por `SessionAuthenticationService.java`. Autentica credenciales, crea el contexto de seguridad y no emite ningún tipo de token.
 - **Validación de Identidad:** El endpoint `/api/auth/me` (en `UserController.java`) lee directamente de la sesión activa del navegador.
 - **Modelo de Datos:** `AuthResponse.java` expone la información del usuario, pero no incluye campos para credenciales de acceso ni tokens.
 - **Roles:** Autorización completamente funcional mediante *authorities* mapeadas en `CustomUserDetailsService.java`.
 - **Frontend:** El estado de la aplicación en React está acoplado a la persistencia de la sesión local a través de `AuthProvider.tsx` y `AuthModal.tsx`.
-

2. Componentes y Clases Afectadas por el Cambio

Backend

- `SecurityConfig.java`: Cambiará el núcleo de la configuración para permitir un comportamiento *stateless* (sin estado).
- `SessionAuthenticationService.java`: Dejará de ser el mecanismo central de persistencia de sesión.
- `UserController.java`: Adaptación de los endpoints de login, logout y `/me` para devolver o validar tokens.
- `AuthResponse.java`: Se extenderá para soportar *access token*, *refresh token* y tiempos de expiración.
- `CustomUserDetailsService.java`: Su propósito cambiará a proveer la identidad requerida para la validación del JWT entrante.

Frontend (React)

- `AuthProvider.tsx` / `authStorage.ts` / `AuthModal.tsx`: Cambiarán su lógica para almacenar, leer y renovar los tokens en el navegador.
 - `ProtectedRoute.tsx`: Adaptación de los guardianes de ruta para verificar la validez del token y los roles del usuario.
-

3. Estrategia y Recomendación de Convivencia Técnica

Para realizar una migración segura y evitar la ruptura del sistema, se aplicará un enfoque de convivencia de dos capas en fases:

1. **Fase 1 (Backend Compatible):** Mantener el flujo de sesiones operativo como respaldo (*fallback*). Introducir la lógica de JWT en paralelo, permitiendo al backend reconocer ambos modos según la configuración o el entorno.
 2. **Fase 2 (Pruebas de Bloqueo):** Validar mediante tests que los nuevos endpoints JWT gestionan correctamente el login, la autorización y el rechazo de accesos no autenticados sin alterar la capa de sesión clásica.
 3. **Fase 3 (Adaptación del Frontend):** Modificar React para migrar al flujo de *Token Bearer*.
 4. **Fase 4 (Limpieza):** Retirar de forma progresiva la infraestructura antigua de sesiones una vez que el flujo JWT esté 100% estabilizado en producción.
-

4. Diseño de Paquetes y Fases de Ejecución

Estructura de Paquetes Propuesta (Backend)

- `com.cursosonline.backend.security.jwt`: Contendrá `JwtService.java` (generación/validación) y `JwtAuthenticationFilter.java` (filtro de petición).
- `com.cursosonline.backend.config.jwt`: Contendrá `JwtProperties.java` (lectura segura de claves desde properties).
- `com.cursosonline.backend.dto.auth`: Contendrá los nuevos contratos de entrada y salida (`AuthRequest`, `AuthTokenResponse`).
- `com.cursosonline.backend.services.auth`: Contendrá `JwtAuthenticationService.java` para orquestar el login con tokens.

Mapa de Ruta Estructurado en Fases

- **Fase 0:** Preparación de la rama de Git y definición de variables en `application.properties`.
 - **Fase 1:** Creación de la capa JWT aislada (servicios y propiedades) sin conectar a la red.
 - **Fase 2:** Integración del filtro en `SecurityConfig.java` habilitando la doble vía (Sesión + JWT).
 - **Fase 3:** Creación de los nuevos DTOs y adaptación de los controladores en `UserController.java`.
 - **Fase 4:** Despliegue del servicio de autenticación y pruebas de validación en paralelo.
 - **Fase 5:** Endurecimiento de seguridad (CORS, control de errores 401/403) y paso final a *Stateless*.
-

5. Checklist Operativo Diario y Criterios Go/No-Go

Esta guía de ejecución permite avanzar de forma incremental, garantizando que el sistema no sufra regresiones funcionales mientras conviven el modelo de sesión y JWT.

Día 1: Base JWT Aislada

- **[] Preparar configuración y contrato técnico:** Definir variables de entorno (secretos, expiraciones) en `application.properties`.
 - **Go:** Parámetros configurados externamente; backlog actualizado; sin impacto en el entorno de ejecución.
 - **No-Go:** Secretos escritos directamente en el código Java o ambigüedad en los tiempos de expiración.
 - *Evidencia mínima:* Archivo de propiedades configurado y parámetros validados.

- [] **Crear estructura de paquetes y clases JWT:** Desarrollar `JwtService`, `JwtAuthenticationFilter` y `JwtProperties`.
 - **Go:** Clases creadas con responsabilidades separadas; el proyecto compila limpiamente.
 - **No-Go:** Acoplar lógica de negocio de login dentro del filtro o generar dependencias circulares.
 - *Evidencia mínima:* Compilación del backend con éxito (**BUILD SUCCESS**).
- [] **Validar integridad del flujo de sesión actual:** Probar el comportamiento de los endpoints activos.
 - **Go:** El login tradicional y el endpoint `/api/auth/me` responden de forma idéntica a la inicial.
 - **No-Go:** Cualquier fallo o cambio en los códigos de estado HTTP en las peticiones de sesión.
 - *Evidencia mínima:* Test manual satisfactorio de login por sesión en el navegador o cliente API.

Día 2: Convivencia Segura en SecurityConfig

- [] **Integrar filtro JWT en la cadena de seguridad:** Registrar el componente en `SecurityConfig.java`.
 - **Go:** Filtro JWT posicionado antes de la autenticación estándar; política de sesión en **IF_REQUIRED**.
 - **No-Go:** Cambiar a política *stateless* antes de tiempo o bloquear peticiones sin token que tienen sesión activa.
 - *Evidencia mínima:* Peticiones con cookie de sesión y peticiones con Bearer Token son válidas simultáneamente.
- [] **Verificar la autorización basada en roles:** Validar los accesos restringidos.
 - **Go:** Los perfiles de administración, profesorado y estudiantado mantienen sus restricciones de acceso intactas.
 - **No-Go:** Respuestas de acceso denegado (403) inesperadas para usuarios con permisos correctos.
 - *Evidencia mínima:* Matriz de verificación de endpoints y roles completada con éxito.
- [] **Consolidar el control de errores HTTP:** Asegurar respuestas semánticas homogéneas.
 - **Go:** El sistema devuelve estrictamente **401 Unauthorized** si no hay identidad y **403 Forbidden** si no hay permisos.
 - **No-Go:** Respuestas erróneas mezcladas o respuestas vacías según el origen de la autenticación.
 - *Evidencia mínima:* Verificación de respuestas en el manejador global de excepciones.

Día 3: Contrato JWT en API y Compatibilidad

- [] **Definir Objetos de Transferencia de Datos (DTOs):** Crear el paquete `dto.auth` con los nuevos contratos.
 - **Go:** Modelos claros para peticiones y respuestas JWT; mantenimiento temporal del contrato anterior.
 - **No-Go:** Exponer o reutilizar la entidad de persistencia `Users` directamente en el cuerpo de la petición.
 - *Evidencia mínima:* Nuevas clases DTO integradas sin errores de compilación.
- [] **Implementar el servicio de autenticación JWT:** Desplegar `JwtAuthenticationService` de forma aislada.
 - **Go:** Lógica de emisión de tokens y refresco operativa; el servicio de sesiones sigue funcionando en paralelo.
 - **No-Go:** Eliminar o alterar la infraestructura del servicio de sesión clásico.
 - *Evidencia mínima:* Generación exitosa del primer token simulado mediante pruebas unitarias.
- [] **Habilitar doble vía en controladores:** Adaptar la capa de exposición web.

- **Go:** El endpoint `/api/auth/me` resuelve la identidad del usuario tanto por token como por sesión.
- **No-Go:** Caída del endpoint `/me` en cualquiera de las dos modalidades de acceso.
- *Evidencia mínima:* Batería de pruebas funcionales ejecutada correctamente para ambos flujos.

Día 4: Endurecimiento (Hardening) y Criterio de Transición

- **[] Auditoría de seguridad operativa:** Revisión final de parámetros críticos.
 - **Go:** Expiraciones temporales estrictas (token de acceso corto, refresco largo); políticas CORS restringidas.
 - **No-Go:** Uso de credenciales por defecto, secretos débiles o políticas CORS excesivamente permisivas.
 - *Evidencia mínima:* Archivo de configuración validado bajo criterios de seguridad empresarial.
- **[] Evaluación de la fase de convivencia:** Comprobación del estado general del backend.
 - **Go:** El backend es robusto, no presenta regresiones y tolera ambos tipos de clientes simultáneamente.
 - **No-Go:** Persistencia de comportamientos indeterminados o fallos intermitentes en la cadena de seguridad.
 - *Evidencia mínima:* Informe de estado de compilación definitivo y confirmación del backend compatible.
- **[] Definición del criterio de desconexión (Fase Stateless):** Establecer las pautas de cierre del flujo antiguo.
 - **Go:** El cliente en React consume JWT y gestiona el refresco de sesión de forma autónoma.
 - **No-Go:** Frontend dependiente de la cookie de sesión del servidor para operaciones críticas.
 - *Evidencia mínima:* Validación completa del flujo de la interfaz de usuario contra la API protegida por JWT.

Semáforo de Control de Riesgos

-  **Verde (GO):** El proyecto compila sin advertencias, los flujos previos de sesión se mantienen totalmente operativos, los componentes de seguridad JWT se integran de forma incremental, los roles operan bajo la política definida y las excepciones HTTP conservan su semántica técnica.
-  **Amarillo (GO CONDICIONADO):** El sistema compila y opera de forma funcional, pero se identifican de uno a dos riesgos menores de integración no bloqueantes. Se requiere documentar el plan de mitigación inmediato antes de continuar con la siguiente actividad.
-  **Rojo (NO-GO):** Se detecta una regresión en los endpoints core existentes (`login`, `/me`), inconsistencia en la aplicación de roles, alteración del filtro global de Spring Security o vulnerabilidades expuestas en la configuración del token. Se detiene el avance hasta solventar el conflicto.